BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

FRAMEWORK FOR ANALYSIS OF CHANGES IN DATA STRUCTURES OF CORE ROUTERS

FRAMEWORK PRO ANALÝZU ZMĚN V DATOVÝCH STRUKTURÁCH PÁTEŘNÍCH SMĚROVAČŮ

MASTER'S THESIS
DIPLOMOVÁ PRÁCE

AUTHOR **Bc. MARIE BEDNÁŘOVÁ**
AUTOR PRÁCE

SUPERVISOR **Ing. JIŘÍ MATOUŠEK, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2024

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Master's Thesis Assignment

145368

Institut:	Department of Computer Systems (DCSY)
Student:	**Bednářová Marie, Bc.**
Programme:	Information Technology and Artificial Intelligence
Specialization:	Computer Networks
Title:	**Framework for Analysis of Changes in Data Structures of Core Routers**
Category:	Networking
Academic year:	2023/24

Assignment:

Literature:

- Dle pokynů vedoucího práce.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Matoušek Jiří, Ing., Ph.D.**
Head of Department:	Sekanina Lukáš, prof. Ing., Ph.D.
Beginning of work:	1.11.2023
Submission deadline:	17.5.2024
Approval date:	30.10.2023

Abstract

Core routers have to be able to work at high speed to keep up with the demands of new Internet services and applications. One of the factors is the classification algorithm utilized in a router in the process of forwarding incoming packets based on their destination IP addresses. Each address is provided to the Forwarding Information Base (FIB) table, which implements the Longest Prefix Matching algorithm (LPM). The FIB table stores prefixes which represents reachable networks. Based on the provided IP address, the FIB table can decide in which way the packet should continue to reach the final destination. There are many LPM algorithms, and each can provide different properties, such as search speed, storage requirements, complexity of updates, etc. The FIB table is constructed from Routing Information Base (RIB) and during the operation of the router, both structures are updated based on the routing information exchanged between routers. In this sense, the thesis focuses on how the data structures of the FIB tables are changed in core routers.

The thesis proposes a benchmark framework that helps to show how different LPM algorithms behave in the context of changes of the FIB data structures. The benchmark is done by simulation, where the LPM algorithm is put in the simplified router model. Then, using the Border Gateway Protocol (BGP) messages, the algorithm modifies the FIB data structure. The whole simulation is monitored and the effects of changes are stored. At the end of the simulation, statistical results are generated. Finally, the framework provides a functionality to change the LPM algorithm and other simulation parameters. The final result is then verified by multiple experiments.

Abstrakt

Páteřní směrovače jsou síťová zařízení, která v kontextu výkonnosti musejí držet krok s požadavky nových internetových služeb a aplikací. Jeden z faktorů, který výkonnost směrovače ovlivňuje, je klasifikační algoritmus, který je součástí procesu přeposílání příchozích paketů na základě jejich cílové IP adresy. Každá adresa je předložena Forwarding Information Base (FIB), která implementuje algoritmus Longest Prefix Matching (LPM) - hledání nejdelšího shodného prefixu. FIB tabulka obsahuje prefixy všech dosažitelných sítí z daného směrovače a na základě poskytnuté IP adresy pak rozhodne, kam daný paket dále přeposlat, aby dorazil na místo určení. Existuje několik LPM algoritmů s různými vlastnostmi, jako je rychlost vyhledávání, náročnost na paměť, náročnost na aktualizaci, a další. FIB tabulka se v průběhu provozu směrovače aktualizuje na základě změn ve Routinf Information Base (RIB). Tyto změny jsou prováděny na základě směrovacích informací, které si mezi sebou směrovače vyměňují. Z těchto poznatků vychází téma této práce, které se věnuje tomu jak, dynamické jsou změny datových struktur FIB tabulek v páteřních směrovačích.

Tato práce se věnuje návrhu a implementaci frameworku, který je možné použít jako pomocný nástroj pro vyhodnocování LPM algoritmů, na základě toho, jak daný algoritmus mění datové struktury FIB tabulek v páteřních směrovačích. Test je prováděn pomocí simulace, kdy LPM algoritmus je nejprve umístěn do zjednodušeného modelu směrovače jako implementace FIB tabulky. Poté, na základě zpráv protokolu BGP (Border Gateway Protocol), bude algoritmus aktualizovat datovou strukturu FIB tabulky. Celá simulace je monitorována a účinky změn jsou zaznamenávány. Na konci simulace jsou poskytnuty výsledné statistiky. Framework dále umožňuje změnit implementaci LPM algoritmu a také nastavení samotné simulace. Nakonec je funkčnost frameworku prověřena na základě experimentů.

Keywords

Router, BGP, framework, statistics, Routing table, RIB, Forwarding table, FIB, LPM, Binary Trie, Tree Bitmap

Klíčová slova

Router, BGP, framework, statistiky, Směrovací tabulka, RIB, Přepínací tabulka, FIB, LPM, Binary Trie, Tree Bitmap

Reference

BEDNÁŘOVÁ, Marie. *Framework for Analysis of Changes in Data Structures of Core Routers*. Brno, 2024. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Jiří Matoušek, Ph.D.

Rozšířený abstrakt

Tato diplomová práce se zabývá zkoumáním změn datových struktur v páteřních směrovačích. Od doby, kdy byl v devadesátých letech, dvacátého století Internet otevřen široké veřejnosti, se přenosová rychlost v páteřních sítích zvýšila z jednotek kilobitů, na stovky gigabitů. Tento nárůst napomohl k zavedení nových technologií a služeb. Avšak vytvořil i tlak na výkonnost a rychlost páteřních směrovačů. Zejména pak na rychlost operace přeposílání, která rozhoduje, kam má být paket odeslán dále, aby dorazil ke svému cíli. Tato operace probíhá na základě informací uložených v tabulce Forwarding Information Base (FIB tabulka), která je generována z obsahu tabulky Routing Information Base (RIB tabulka). S pomocí FIB tabulky a cílové adresy paketu pak operace řeší úlohu vyhledání nejdelšího shodného prefixu (Longest Prefix Matching problem, LPM). Rychlost, s jakou již nyní musí být přeposílání prováděno, vedla k implementaci operace přímo v hardwaru. Přesto mohou být celkový výkon a rychlost omezeny kvůli změnám, které se do FIB tabulky průběžně propagují z RIB tabulky na základě přicházejících směrovacích aktualizací.

Cíle této práce jsou navrhnout, implementovat a demonstrovat framework, který bude analyzovat změny v datových strukturách LPM algoritmů v páteřních směrovačích. Dále bude výsledný framework umožňovat definování datové struktury LPM algoritmu. Pro analýzu budou využita data protokolu BGP (Boarder Gateway Protocol), který je použit pro zasílání směrovacích informací mezi páteřními směrovači. Jako výstup této analýzy bude sada statistických dat. Dále bude framework poskytovat dva již implmenetované LPM algoritmy. Dalším cílem je pak demonstrovat funkčnost frameworku na vybraných BGP datech a LPM implementacích v experimentech. Na základe těchto experimentů pak budou vyvozeny závěry, zda framework opravdu umožňuje zachytit a vizualizovat dynamické chování ve strukturách FIB tabulek. Posledním krokem pak bude návrh možných dalších rozšíření.

První kapitola je věnována základům architektury TCP/IP, jejímu konceptu a terminologii. TCP/IP představuje pětivrstvý model. Z daných pěti vrstev se tato práce nejvíce věnuje vrstvě síťové a aplikační. Tyto vrstvy obsahují protokoly, které jsou klíčové pro směrovače a pro operace směrování a přeposílání. První z těchto protokolů je Internetový protokol IP. V rámci tohoto protokolu jsou také popsány dvě nejvíce rozšířené verze, IPv4 a IPv6. V rámci aplikační vrstvy je popsán protokol BGP, který je používán jako směrovací protokol využívaný páteřními směrovači.

Druhá kapitola se věnuje algoritmům, které implementují operaci hledání nejdelšího shodného prefixu (LPM algoritmy). Prvním představeným LPM algoritmem je Binární Trie používající strukturu jednoduchého binární stromu. Tato struktura se dá jednoduše aktualizovat, avšak může obsahovat řídké části (dlouhé sekvence uzlů, které se musejí projít pokaždé, když se přistupuje k prefixům na nižších úrovních). Způsobenou nekompaktnost dlouhých sekvencí uzlů u Binární Trie řeší algoritmy Multibitové Trie, které oproti Binární Trie procházejí strukturou ne na základě jednoho bitu v jednom kroku, ale po více bitech. Tyto algoritmy používají složitější uzly, které mohou pokrývat celé podstromy původní Binární Trie. Jedním z nejefektivnějších multibitových algoritmů je Tree Bitmap algoritmus.

Na základě prvních dvou kapitol se třetí kapitola věnuje návrhu a architektuře výsledného frameworku. SandRouter se skládá z několika modulů, které dohromady představují zjednodušený směrovač. Pro možnost analýzy chování implementace FIB tabulky provádí SandRouter simulaci provozu, tvořeného BGP aktualizacemi. Po dobu simulace se shromažďují informace o změnách, které byly provedeny v datové struktuře FIB tabulky. Na konci simulace se vygenerují výstupní statistiky. Data, která jsou po dobu simulace shro-

mažďována, byla vybrána na základě implementace algoritmu Binární Trie. Výsledné statistiky byly rozděleny do dvou skupin, a to Dynamické statistiky a Strukturové statistiky. Pátá kapitola se následně věnuje implementaci a validaci frameworku podle návrhu. SandRouter poskytuje dvě implementace FIB tabulky, jednu pomocí algoritmu Binární Trie a druhou pomocí Tree Bitmap algoritmu. Framework podporuje dvě verze IP protokolu, tedy umožňuje zpracovávat prefixy síťových adres verzí IPv4 a IPv6. Spolu s dalšími pomocnými objekty a metodami kapitola popisuje i možnost implementace uživatelem definované FIB tabulky.

Další částí této práce bylo otestovat a demonstrovat, že je SandRouter schopný vizualizovat a zachytit dynamičnost ve strukturách různých implementací FIB tabulek. Byly provedeny tři experimenty. První experiment ukázal, že analýzou celé struktury FIB tabulky se neprojeví zřetelná dynamičnost. Navíc výstupy Strukturních Statistik byly velmi podobné, ať už byla provedena simulace dat pocházejících z jakéhokoliv BGP kolektoru, z kteréhokoli regionu nebo kontinentu. Na základě výsledků prvního experimentu bylo pro druhý experiment rozhodnuto, že budou vstupní data rozdělena podle Regionálních Internetových Registrátorů. Díky tomuto rozdělení bylo dále možné vybrat pět různých kolektorů z pěti různých oblastí, kde každá byla spravována jiným registrátorem. Výsledky potvrdily, že rozdělením provozu je možné lépe zachytit změny ve strukturách FIB tabulek. Nejvíce je bylo možné vidět při použití implementace Tree Bitmap algoritmu. Navíc byly zachyceny odlišnosti ve velikostech a zaplnění FIB tabulek obsahujících prefixy stejného registrátora, ale v jiné oblasti. Poslední experiment se věnoval prefixům verze IPv6. Výsledky, podobně jako v prvním experimentu, neukázaly zřetelnější dynamičnost, dokud nebyla použita implementace Tree Bitmap algoritmu.

Výsledkem této diplomové práce je framework SandRouter, který je možné použít pro analýzu změn datových struktur v páteřních směrovačích. Framework prokázal schopnost vizualizovat změny ve strukturách, pokud se zaměří na menší část celkové FIB tabulky v páteřním směrovači.

Framework for Analysis of Changes in Data Structures of Core Routers

Declaration

I hereby declare that this Master's thesis was prepared as an original work by the author under the supervision of Ing. Jiří Matoušek, Ph.D.. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis.

.....
Marie Bednářová
May 15, 2024

Contents

List of Figures

Chapter 1

Introduction

In our world, the most monumental architecture might be the one that remains unseen. However, its presence is ubiquitous; the Internet. This global network of networks connects cities, states, and continents. It allows sending messages that reach their destination in seconds, at most a few minutes. It helps users connect through social networks, share their experiences, stream their daily lives or events, and provide a large amount of information. All of these functions have been added to the Internet throughout its existence since it was opened to the public in 1991.

As mentioned here [1], the first connection between the Czech Republic's national network and EARN (European Academic Research Network) in 1991, the data transfer rate provided was 9,6 kb/s. With the establishment of the CESNET group and the TEN-34 CZ network, the Internet became more available not only to universities and research, but also for the general public. So far, the Czech Republic's core network is capable of transferring data at a speed of 400 Gb/s[7]. One could ask what changes have been made that have helped to grow the speed of the Internet. One of them is the evolution of the router architecture.

A router, a so-called *network layer device*, helps route *packets* from a source to a destination. This device can be found at the edge of a network, where it acts as a gateway for incoming and outgoing packets. All of these streams will need to pass through this device. The router decides which way to send the packet. This decision is made based on the information processed by *routing algorithms*, which then provide routing paths in the form of the *Routing Information Base* (RIB). Based on this RIB, the router builds a second structure, *Forwarding Information Base* (FIB). This table is used in the operation called *forwarding*.

FIB table provides a fast way to decide which interface the packet should be sent to reach its destination. It is implemented by the *Longest Prefix Match* (LPM) algorithm, which as the input uses the destination IP address of the packet. Based on the address provided, a classification process is performed on the FIB data structure. There are many LPM algorithms, and each can use a different approach to store information, update data, or the way of search. One of the groups of these algorithms use tree structures, which are further studied in the thesis. The reason why is, that at the software level, the FIB table would be represented using some form of tree. However, in hardware, the utilized data structure may be distributed over multiple memory blocks. This is a problem for routing processes that need to access different parts of the memory in a short time. Therefore, forwarding causes a performance bottleneck. The overhead of updating the FIB makes the overall performance even worse.[10]

The requirements arising from core (i.e., high-speed) networks and the ever increasing speed had an effect of implementing packet classification in core routers solely on hardware. However, the critical moment for the speed and performance of the router can be in situations where the structure of the FIB table is modified due to routing updates [24]. In this sense, the goal of the thesis is to design and implementat a framework that can provide benchmark testing of LPM algorithms. The test should verify how the LPM algorithm changes the data structures of the FIB tables and how dynamically these changes are made. The following Chapter 2 introduces the background mechanism and terms of TCP/IP architecture, with important terminology. The chapter introduces important protocols such as the Internet Protocol (IP), including IPv4 and IPv6. The BGP protocol (Boarder Gateway Protocol) is also introduced, which is a routing protocol used by core routers. In Chapter 3, classification algorithms that use Longest Prefix Matching are introduced. Two groups of algorithms are described: the Binary Tries and the Multibit Tries. Based on the requirements, two of these algorithms are further implemented in the final framework.

Based on the theory introduced in the first two chapters, the design and architecture of the framework are described in Chapter 4. The chapter contains a description of statistics, based on which the LPM algorithms are evaluated. The implementation of the proposed design is described in Chapter 5. The framework supports two versions of the IP protocol. It also provides two already implemented LPM algorithms, introduced in Chapter 3. There are also descriptions of validation of the implementation, main methods, and objects. The customization is also described, which was the requirement for the proposed framework. Customization provides a way to change the implementation of the LPM algorithm and other parameters of the test simulation.

The final part is to demonstrate and verify that the final framework is capable of visualizing the dynamicity of changes in the data structures of the core routers. In Chapter 6 are described three experiments with results. Finally, the thesis is summarized in Chapter 7, also with the introduction of possible improvements.

Chapter 2

TCP/IP Architecture

Each device - including routers - that wants to communicate through the Internet needs, apart from the appropriate connection to the network, to know the format of sent and received messages. Communication protocols define these formats, rules, and mechanisms. This chapter describes the model on which the Internet is built. First, the meaning of basic terms is described. After this, the first standardized reference model for computer network design is introduced. This model is a basic concept for the field of computer networks. The next part describes the model used in modern networks. With this model, there is a large group of related protocols. However, because this group is too broad to cover, only those related to the thesis are mentioned. These are the IP protocol, IPv4 and IPv6, and BGP.

Definition 2.0.1 (Protocol)**.** *Protocol defines the format and the order of messages exchanged between two or more communication entities, as well as the actions taken on the transmission and receipt of a message or other event.[22]*

Protocols of TCP/IP Architecture describes communication between two elements (also known as *peer-to-peer protocols*[14]). The requirements for peers to communicate with each other are as follows;

- to know the protocol
- be placed in the same *layer*

Because a computer network can be a complex system, engineers divided the general view of the system into layers. Each layer then provides special functionality in communication between peers. The network can then be modeled as a top-down layered system. At each layer, different protocols operate. Each layer is implemented using hardware, software, or a combination. In addition, each layer has special functionalities that are provided to the higher layers and can use services from the lower layers. This concept is also known as the *service model*. The layers are modular and are isolated from each other. Therefore, the changes can be done efficiently. If there is a need to change or update one layer, it could be done without the need to change the other layers[22]. Today, all of these approaches are combined into one standardized and publicly approved model.

2.1 ISO/OSI Reference Model

In the early years of information technology, multiple models were developed to design protocols and schemes for computer networks. But only one was the most successful.

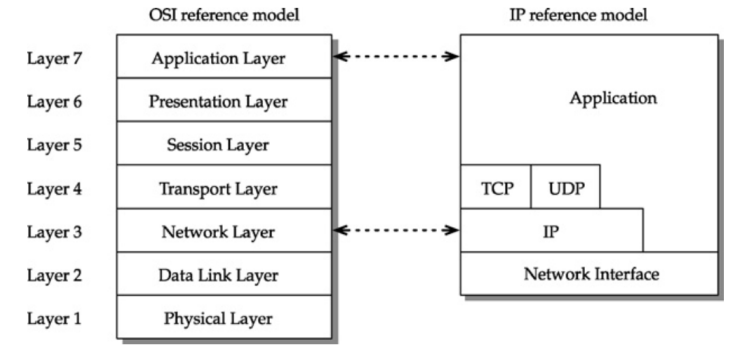

Figure 2.1: Seven-layered ISO/OSI model and four-layered TCP/IP model.

In 1978 the Open System Interconnection model (OSI model) was standardized by the International Organization of Standardization (ISO). The OSI model is defined in ISO/IEC 7498-1 and is also known as ITU-T Recommendation X.200 (or simply ISO/OSI model)[20]. The ISO/OSI reference model proposed the architecture that divides a network communication system into seven abstract layers (see Figure 2.1). Each layer has its name and the predefined functionalities and services that it provides. The model then describes how the layers divide the workload during sending or receiving data and how the layers cooperate. It also defines the names of the communicating endpoints and terms used for the data units on each layer [22]. However, ISO/OSI is much more complex than what is needed for Internet applications.

2.2 TCP/IP Model

As mentioned in the previous paragraph, the ISO/OSI model defines seven modular protocol layers that together create the basic scheme of a computer network system. But for the Internet and most network applications, this approach is too complicated. There is no need to develop applications with a whole new stack of protocols. It is effective to use technologies already implemented and prepared. Or omit some of them. In this sense, the TCP/IP protocol stack was designed. From the start, it points out the elementary protocols which are most likely used for communication on the Internet: Transmission Control Protocol (TCP) and Internet Protocol (IP). TCP/IP provides a suite of protocols that govern the way data are transferred and received through networks, including the Internet. The model consists of five layers, simplifying the previous seven-layered ISO/OSI model. From bottom to top, the layers are called: *physical layer*, *data link layer*, *network layer*, *transport layer*, and *application layer*.

Each of the mentioned layers (mentioned in the previous paragraph) contains relatively independent protocols. Depending on the type of service, device, and network application, as mentioned in [14], different protocols could be used for each layer. In addition, not all types of network systems need to implement all five TCP/IP layers. In this sense, devices using the TCP/IP protocol stack can be grouped into two general groups:

- a *hosts* or *end systems* (ES)
- an *intermediate node* (router) or an *intermediate system* (IS)

Hosts can represent network applications, such as email services, file transfer services, or as an example Domain Name System (DNS). These services implement all five layers. The IS or router represents the device that mostly operates on the network layer, and so it does not usually have to use protocols from higher layers. In the end, the combination of layers and protocols used by each service can differ. The only exception is the Internet Protocol (see Section 2.3.1), which is used by all network systems. In this sense, the part of the IP protocol is described in more detail in the following sections. In general, the two lowest layers are represented as one layer, the *link layer*, and TCP/IP is then called a **four-layer model**. When a programmer develops a new network application, he can rely on these two layers to be already prepared. For this reason, TCP/IP mainly defines the three higher layers and only describes the interface with the link layer[14]. This convention is used in the thesis.

In the context of the thesis, the key layers are only the network layer and the application layer. In this sense, the other two layers will not be mentioned in the following text.

Encapsulation It is convenient to mention some terms used for transmitted data in context of TCP/IP layers, how the data „flows“ through the system, and what is happening with them. It is important to know that two communicating peers (or hosts) are connected only by a physical layer[14]. The entire workflow is shown in 2.2. The data sent by a sender must pass through all lower layers. On the destination system, it needs to go through all layers up to the actual communicating layer. On the sender side, each layer adds additional information to the data, also known as headers. The data package is generally called a *protocol data unit* or PDU. However, the name could differ in the context of each layer[14]. The extra information, provided in *headers*, helps with the PDU transmission through a network (or the entire Internet) and with the right interpretation of the transmitted data by the receiver. This process is called *encapsulation*. The content of headers (source and destination addresses, source and destination ports, checksums, type of protocols, amount of useful data, etc.) depends on the protocol used for communication. In the receiver system, the process is done in the opposite way. The header of the corresponding layer is analyzed and removed, and the remaining PDU is passed to the upper layers. This process is called *decapsulation*.

2.3 Network Layer

Network Layer (also known as the Internet Layer) has the function of delivering data from the source to the destination (end-to-end delivery) on the network. Although the endpoints are not connected by a physical medium. The data transported are called *packets*[14]. The network layer provides services to the transport layer to send segments (which is the name for the data unit from the transport layer). A segment is encapsulated in a packet by adding a header with source and destination addresses (see the scheme of the IP header 2.4). The packet is then passed to the link layer. On the receiving side, the data are decapsulated and the remaining segment is passed to the transport layer[22].

If one endpoint sends data to the other endpoint, the data must be processed by the network layer. This layer does not only consist of the hosting endpoints. On the way to the destination, the packet passes through different network devices, different networks, or even through the entire Internet. In this effect, the destination of the packet could be anywhere on Earth. The navigation of the message through the Internet is provided by routers

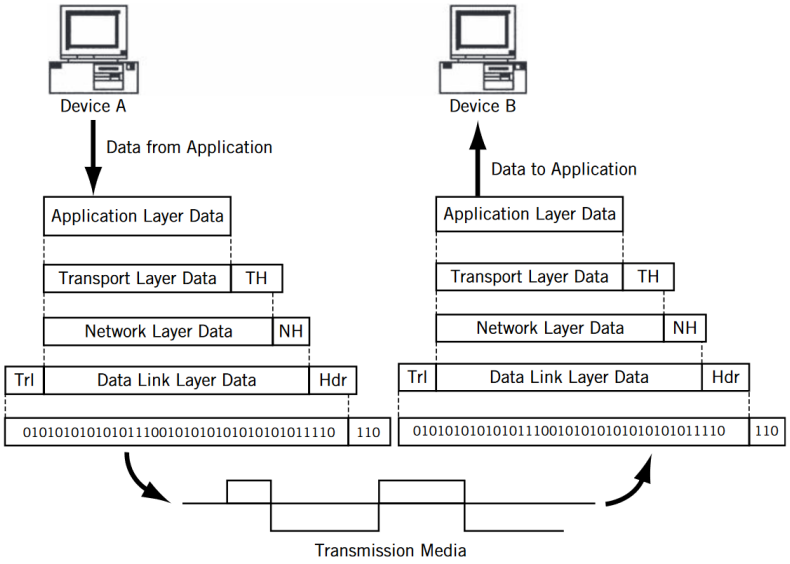

Figure 2.2: The scheme of encapsulation of sent data (left part) and decapsulation of received data (right part)[14].

on the way between the source and destination. A **router** is a device that can decide, based on the destination address in the header of the packet, which way to send the packet to get it to the destination endpoint. Thanks to this support, the functionality of a network layer can generally be split into two subfunctionalities[22]:

Forwarding The function of passing packets from one router to another router, or from host to a router, is called forwarding. The action is also called „*hop*“[14]. The passing action is sending a packet through selected interface of the router. The selected interface is decided on the basis of the special data structure and the destination address contained in the packet header. This structure is stored on each network device, called *forwarding table*. It is also known as *forwarding information base* (FIB)[22].

Routing The network layer needs information on the reachability of different endpoints and hosts on the Internet (or just a local network). On the basis of that, the layer decides which path, made up of network devices, should be taken by the packet. These paths are determined by *routing algorithms* and the resulting paths are saved in *routing tables*, also known as *routing information base* (RIB)[22]. Building these tables is called *routing*. From the resulting routing table, the forwarding table is then built[14].

2.3.1 IP Protocol

The Internet Protocol is a network layer protocol designed to interconnect different communication networks. It provides two basic functions: addressing and fragmentation[2]. Each host on the Internet must be identified with a unique identifier so that the other hosts can communicate with him. The IP protocol defines the rules and mechanisms of how hosts are identified on the Internet. There are two main versions of the addressing schemes defined by the IP protocol. Based on the address, which is then used in the IP header, the network

layer devices can transmit *internet datagrams* toward specified destinations. The selection of the path is called routing 2.3[2].

Fragmentation occurs if a datagram is too large for a network link through which data are to be transferred. In this case, the datagram is split into smaller data packages, and the information about fragmentation is saved in their IP headers. After being delivered to the destination, the fragments are reassembled [2].

The IP protocol model is implemented by modules placed in each host, which communicates through the Internet, and in routers, which interconnect different networks (also known as gateways) [11]. The implementation scheme of the Internet Protocol is shown in Figure 2.3. The protocol defines common rules for the interpretation of address fields in headers and the fragmentation and reassembly of datagrams. For routers, the protocol also defines procedures for making routing decisions and other functions [2].

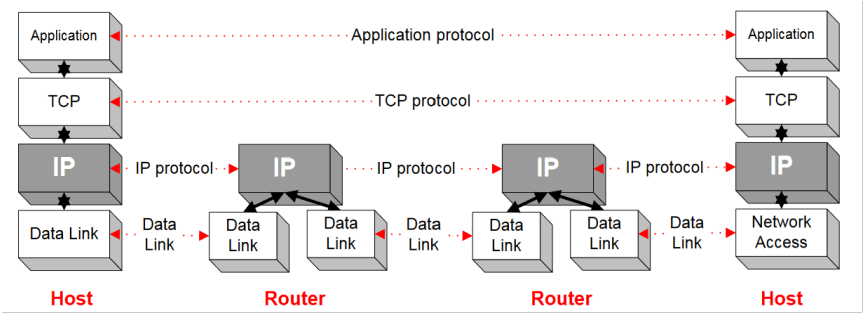

Figure 2.3: The scheme of the Internet Protocol implementation by hosts and routers[11]

The IP protocol is an unreliable, connectionless, best-effort service[11]. Each datagram is treated independently. In this effect, the IP protocol does not recognize logical streams between hosts, which makes it connectionless. If some datagram is lost, the IP protocol does not implement any mechanism to recover it, making it an unreliable protocol[2]. The *best-effort* service means that datagrams are not guaranteed to be received in the order they were sent, not even their delivery is guaranteed. Neither the time of datagram delivery nor their maximum delay is guaranteed. All these services must be implemented by protocols of higher layers (for example, by the Transport Control Protocol)[22]. Otherwise, the service parameters can be indicated using the *Type of Service*[2] field in the IP header.

2.3.2 IPv4

The older version, IPv4, was introduced in RFC 791[2], together with the IP protocol. This scheme assigns to each host a unique 32-bit long value. The value is written in bit format, with the most significant bit (MSB) on the left and the least significant bit (LSB) on the right. The address is a hierarchical value consisting of two parts: the network identifier (*netid*) and the host identifier (*hostid*). The Internet can thus be viewed as the interconnection of networks, where each of them has a unique netid. The network part is also known as the IP prefix. In each of these networks, there is a set of hosts with their unique hostids (in the context of the actual network). To help readability for users, the IPv4 address could be transferred to dotted decimal notation, where the 32-bit value is split into four 8-bit parts, known as octets. Then each octet is shown as a decimal number separated by dots[2].

In Figure 2.4, an example of the general Internet Datagram Header is shown. The IPv4 header consists of fourteen information fields comprising 192 bits (24 bytes). The header format for the IP protocol and the IPv4 protocol is the same. For IPv4, a field Version (shown as a first field of the header in Figure 2.4) will contain the value 4. This field is also an indicator of the header format. Because different versions of the IP protocol use different formats of IP headers (see Section 2.3.3).

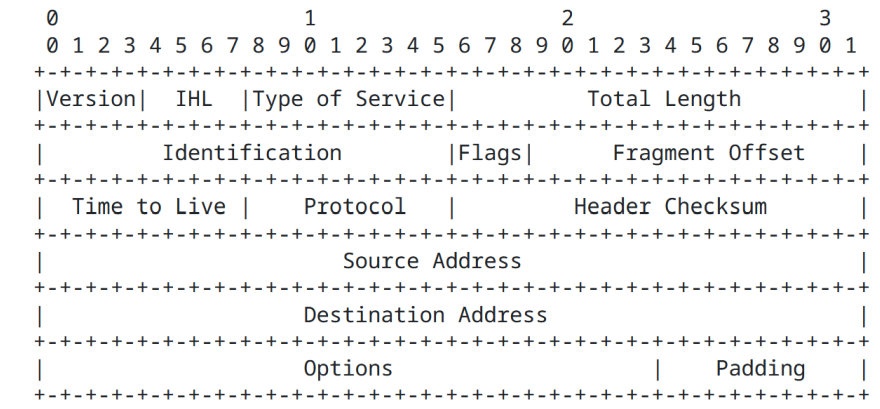

Figure 2.4: The scheme of IP header[2].

Classful Addressing Scheme If there is a general IPv4 address, there should be some information or mechanism to identify the boundary between the network and the host portion of the IP address[14]. As an example, for address 192.168.2.35, there is no way to identify which bits are part of the netid and which ones are part of the hostid. In the original design [2], the *classful addresing* scheme was introduced. Five classes were defined: A, B, C, D, and E. The first three (A, B, and C) were the main classes used for unicast addressing (host-to-host communication). Class D was used (and still is) for IPv4 multicast traffic, and class E was reserved for experimental purposes[14].

The main concept of classful IPv4 addressing, as described in [14], is as follows. The 32 bits of the IPv4 address can be classified by identifying the value of the initial bits. Figure 2.5 shows classes A, B, C, and D, with defined values. After classification, the identification of the netid part and the hostid part can be performed. Each class describes networks of different sizes (based on the value of network bits and host bits from an IP address). This scheme was fixed and did not allow for variability in the size of the networks and better manage the address space. In addition, this led to an overload of routing tables because there were too many small networks that needed to be stored on every core Internet router. The problem escalated in such a way that by 1994 to 1995, the IPv4 address space was almost exhausted. In this sense, new concepts were defined[14].

Classless Interdomain Routing The concept (also known as CIDR) uses a special format of IP addresses, so the information about netid and hostid is recognizable [12]. Here, the network part of the address does not identify a class, but rather a socalled *prefix*. The IPv4 block is provided with an additional value after slash (/). For example, the IP address 192.168.19.48/24 means that a host 48 is in the network with the address 192.168.19.0. The number after the slash is a portion of bits from the left, which are

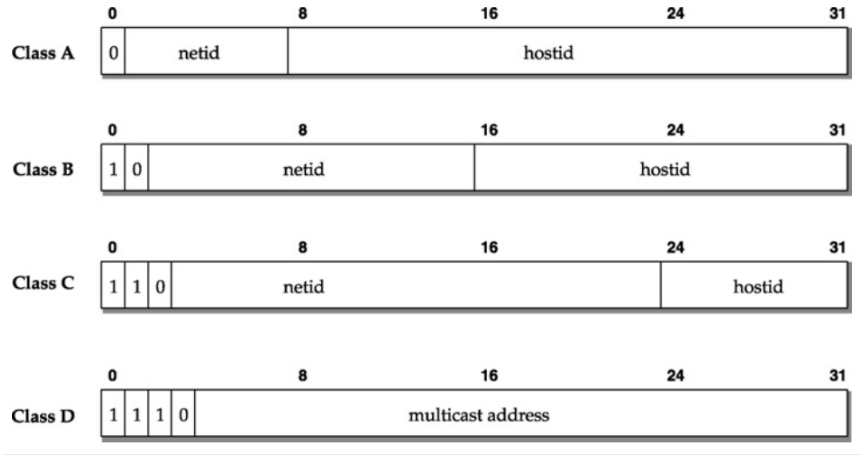

Figure 2.5: The illustration of classful scheme for classes A, B, C, and D with corresponding leading bits[26].

part of the network address. That means the first 24 bits correspond to the first 3 octets. The prefix value for CIDR could be anything from 0 to 32.

The need for better management of the IPv4 address space, as described in [14], was not only about the exhaustion of the IPv4 address space. There was also a rapid increase in the size of the Internet core routing tables. These were needed to handle the many Class C addresses of new users. In that case, CIDR comes up with the assignment of the former classful address space in groups, where each is a block of *contiguous* IP addresses. This CIDR's plan was applied for Class C and allowed a possibility to better manage and redistribute the address space. In this way, it allowed for the creation of networks with a variable number of host addresses. For illustration, Figure 2.6 shows all possible prefix lengths, with the number of classful networks that the prefix represents. The figure does not contain the mask /0, which matches the whole Internet.

2.3.3 IPv6

IPv6 was introduced in [16] and was designed as a successor to IPv4. In the same way as IPv4, IPv6 addresses are used to identify interfaces and sets of interfaces. It brought multiple changes and updates. First, the address capability was expanded to 128 bits. This introduced more levels of addressing hierarchy and a much broader address space than IPv4[16]. Next, IPv6 brings an increase in the size of the IP header (from 192 bits to 320 bits), but with a simpler format. Most of the space is taken up by the Source and Destination fields[16, 14].

The IPv6 addressing model is different from the IPv4 model. The whole definition of the architecture is specified in [15]. The most important parts are mentioned here. An IPv6 address consists of eight groups of numbers; each could be represented as a 16-bit binary number or as four hexadecimal numbers. The groups are separated by colons. Because the resulted address can be too long to use, there are multiple mechanisms to simplify the representation. One of the issues can be long strings of leadign zeros. IPv6 defines the abbreviation mechanism to omit this. The groups of zeros in an IPv6 address are then replaced by two colons ::. This replacement can be performed only once, so the format of the original address will remain unambiguous[14].

Prefix Length	Dotted Decimal Netmask	Number of Classful Networks	Number of Usable IPv4 Addresses
/1	128.0.0.0	128 Class A's	2,147,483,646
/2	192.0.0.0	64 Class A's	1,073,741,822
/3	224.0.0.0	32 Class As	536,870,910
/4	240.0.0.0	16 Class A's	268,435,454
/5	248.0.0.0	8 Class A's	134,217,726
/6	252.0.0.0	4 Class A's	67,108,862
/7	254.0.0.0	2 Class A's	33,554,430
/8	255.0.0.0	1 Class A or 256 Class B's	16,777,214
/9	255.128.0.0	128 Class B's	8,388,606
/10	255.192.0.0	64 Class B's	4,194,302
/11	255.224.0.0	32 Class B's	2,097,150
/12	255.240.0.0	16 Class B's	1,048,574
/13	255.248.0.0	8 Class B's	524,286
/14	255.252.0.0	4 Class B's	262,142
/15	255.254.0.0	2 Class B's	131,070
/16	255.255.0.0	1 Class B or 256 Class C's	65,534
/17	255.255.128.0	128 Class C's	32,766
/18	255.255.192.0	64 Class C's	16,382
/19	255.255.224.0	32 Class C's	8,190
/20	255.255.240	16 Class C's	4,094
/21	255.255.248.0	8 Class C's	2,046
/22	255.255.252.0	4 Class C's	1,022
/23	255.255.254.0	2 Class C's	510
/24	255.255.255.0	1 Class C	254
/25	255.255.255.128	1/2 Class C	126
/26	255.255.255.192	1/4 Class C	62
/27	255.255.255.224	1/8 Class C	30
/28	255.255.255.240	1/16 Class C	14
/29	255.255.255.248	1/32 Class C	6
/30	255.255.255.252	1/64 Class C	2
/31	255.255.255.254	1/128 Class C	0
/32	255.255.255.255	1/256 Class C (1 host)	– (1 host route)

Figure 2.6: CIDR Prefixes and Addressing. The mask /0 matches the entire Internet. This is further discussed in [14].

```
FEDC:BA98:7654:3210:FEDC:BA98:7654:3210

1080:0:0:0:8:800:200C:417A
```

Figure 2.7: Examples of the IPv6 address format[15].

Another feature is the representation of an IPv4 address using the IPv6 address convention. For example, the IPv4 address 10.10.0.1 could be written as the IPv6 address *0:0:0:0:0:0:A0A:1* or in abbreviated form as *::A0A:1*. It could also be left as a dotted decimal as *::10.10.0.1*. This convention is also supported by routers[14].

IPv6 Address Prefixes IPv6 specifies multiple types of addresses, determined by the prefix format. The prefix can be understood similarly to IPv4 address prefixes in the CIDR notation. The general notation of IPv6 is $ipv6/prefix$, where the first part is an IPv6 address (in any notation mentioned in the previous paragraph) and the second part is a decimal value of the number of the leftmost contiguous bits of the prefix[15].

```
12AB:0000:0000:CD30:0000:0000:0000:0000/60
12AB::CD30:0:0:0:0/60
12AB:0:0:CD30::/60
```

Figure 2.8: Examples of the IPv6 prefix format[15].

2.3.4 IP Address Assignment

The management and assignment of IPv4, IPv6, and Autonomous System Numbers (see Section 2.4.1) was for a long time a work of the Internet Assigned Number Authority (IANA). With the growth of the Internet, the Internet Activities Board (IAB) published in [6] the recommended policies and procedures on the distribution of Internet identifier assignments. This led to a decision to distribute the registration function to specific geographic areas. In this way, Regional Internet Registries (RIRs) were created[13]. Today, five RIRs are recognized.

- ARIN (American Registry for Internet Numbers) manages the regions of Canada, Caribbean and North Atlantic islands, and the United States[5]
- RIPE NCC (Reseaux IP European Network Coordination Center) manages the regions of Europe and Middle East[28]
- APNIC (Asian Pacific Network Information Center) manages the regions of Asia and Oceania[4]
- LACNIC (Latin American and Caribbean Network Information Center) manages regions of Latin American and Caribbean territories[23]
- AFRINIC (African Network Information Center) manages the regions of Africa[3]

As described in [13], the IPv4 address space was divided by the Internet Registry (IR). The IANA remains the default manager of those address blocks that were not delegated to anyone. In addition, IANA manages all reserved address blocks. The list of IPv4 address space allocation is provided in the IANA IPv4 Address Registry[18]. The list of IPv6 unicast address space allocation is provided in IPv6 Global Unicast Address Assignments[19].

2.4 Application layer

This layer is the highest in the entire TCP/IP model. All protocols and services within this layer use services from the lower layers. An application running on the end systems identifies hosts at the Application Layer. This layer is responsible for preparing the data that are processed by the lower layers. Therefore, the data are delivered to their destination, properly processed and interpreted, and finally provided to another host.

In the context of the thesis, an example application layer protocol is described. This protocol is specific because it is used by devices operating on the network layer. But for routing between huge networks, it is convenient and crucial to use more reliable mechanisms. In this way, these devices need to be more complex to use the Traffic Control Protocol (TCP). The following protocol then helps connect users across the entire Earth.

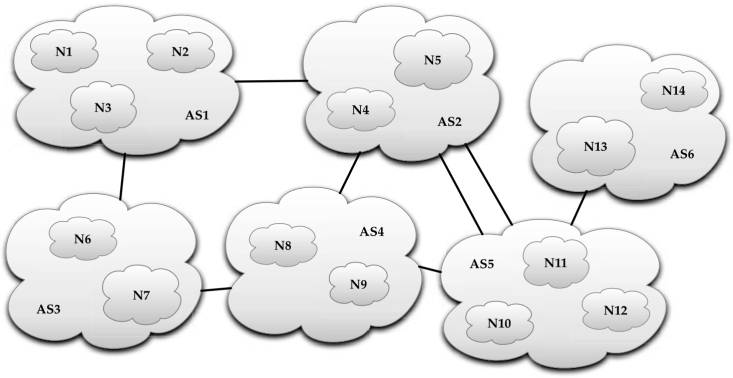

Figure 2.9: The simplified view at the Internet as graph of autonomous systems, connected by BGP sessions[25].

2.4.1 BGP

The Border Gateway Protocol is one of the routing protocols. It is used to communicate information about networks in *autonomous systems* (AS) with other ASes. An AS could be seen as a group of smaller networks. These networks are typically under the same administration, but this is not a strict rule. To distinguish autonomous systems, a 16-bit value identifies each of them; see Figure 2.10 for an example. The value ranges from 0 to 64511; the remaining values are reserved for private AS numbers[14]. The role of BGP is to share information about networks within the AS with other ASes on the Internet[14]. In the context of BGP, a network is understood as an IP *prefix-defined network*, see the content inside each cloud in Figure 2.9. Next, the term *route* in the BGP specification[27] is described as a unit of information that pairs a set of destinations with the attributes of a path to those destinations. The set of destinations are systems whose IP addresses are contained in one IP address prefix carried in the Network Layer Reachability Information (NLRI) field of an UPDATE message (see Figure 2.4.1). An example could be a route (AS5, AS4, AS3, AS1) to network N1 from the point of view of AS6, see Figure 2.9[25].

Each router that participates in the communication with BGP is called a BGP *speaker*. BGP speakers must be routers that could establish a Transport Control Protocol (TCP) session. All the information in the BGP is exchanged through this reliable connection because it is crucial for the overall routing between ASes. The BGP speakers are on the edge of an AS, peering with neighboring BGP devices from different ASes (for illustration, see routers R4 and R2 in Figure 2.10). This group of routers also provides the so-called *exterior BGP* (EBGP) connection. In this way, the routers of one AS are getting information about neighboring ASes. To share this information within the AS, the *internal BGP* (IBGP) connection is used. During sessions, the peers periodically exchange messages, so that both sides know that the other side is still „alive“. If the session fails (the peer stops responding or sends the NOTIFICATION message, see Figure 2.4.1, to close the session), another peer must stop using the information introduced by the peer with whom the connection was lost. The rules for the creation of routing tables and the exchange of information in BGP follow a *path vector* routing protocol approach[25].

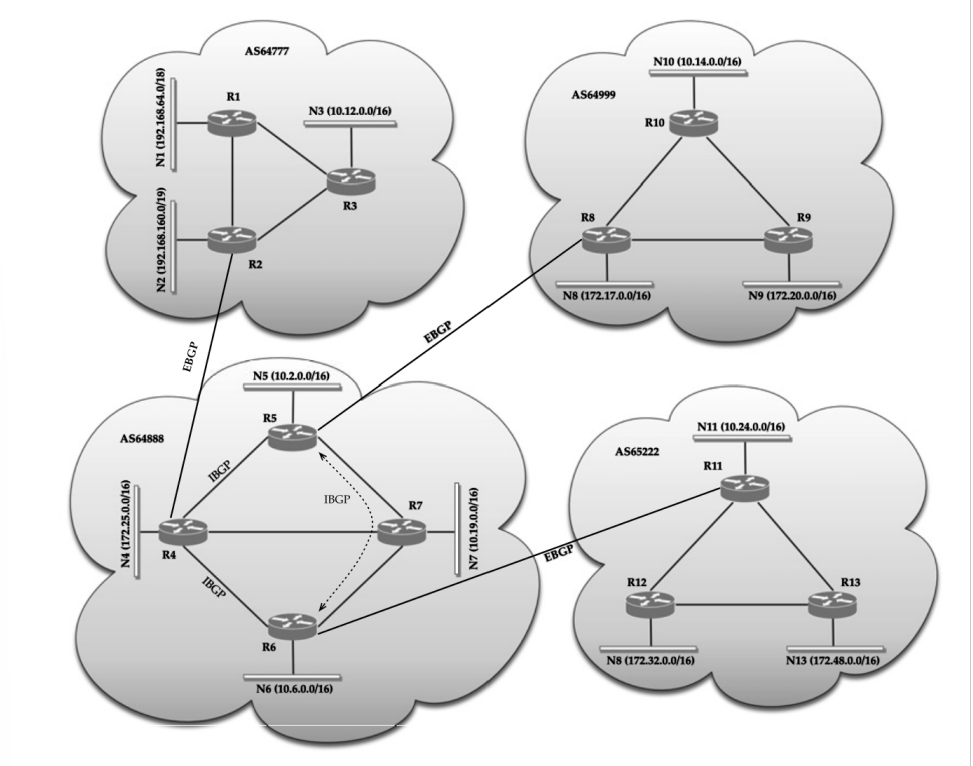

Figure 2.10: The scheme of a BGP system over multiple autonomous systems. There are also visualized examples of external BGP and internal BGP[25].

BGP Messages BGP uses a special message format, as explained in the BGP specification [27]. It is made of a BGP header and fields of data according to the type. There are four types of BGP messages:

- **OPEN** After the TCP connection is established between two BGP speakers, each side sends the OPEN message. With this message, peers introduce themselves, such as what AS they belong to, what version of BGP they know, and more. If the peer accepts the OPEN message, it responds with a KEEPALIVE message.
- **UPDATE** This type of message is used to transfer routing information between BGP peers. Using these messages, each BGP speaker creates and updates its *Adj-RIBs-In* tables, based on which then the Decision Process within the router evaluates which routes are stored or updated further. The messages are used to advertise routes that share common path attributes or to withdraw multiple unfeasible routes. All of this information can be sent together in one UPDATE message.
- **KEEPALIVE** This message is used to determine if connected peers are reachable. The KEEPALIVE message is sent periodically by peers within the Hold Timer interval. If no KEEPALIVE message came within the interval, it could mean that the peer has some kind of failure or that there is a failure on the route between peers. In that situation, a BGP connection must be terminated.
- **NOTIFICATION** This type of message is used to acknowledge that an error was detected in one of the peers in a session. The BGP session is terminated immediately

after the message is sent. Multiple types of error can occur in the BGP speaker (see [27]).

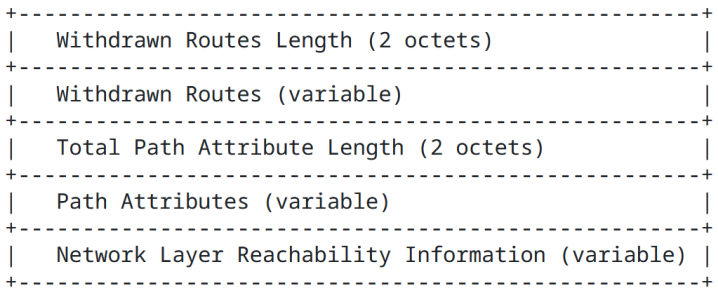

Figure 2.11: Fields of an UPDATE message, based on the BGP specification[27].

UPDATE Message Format In the context of the thesis, the UPDATE message is the most crucial of all BGP messages. As mentioned in the previous paragraph, the information within these messages is used to create and update RIB structures inside routers. Figure 2.11 shows the fields that may be present in the UPDATE message. The first two fields correspond to the routes that were determined to be unreachable by the peer, from which the message is sent. The **Withdrawn Routes Lenght** contains a 2-octet value, which indicates the total length of the Withdrawn Routes field. If this value is set to 0, the Withdrawn Routes field is not present in the message. **Withdrawn Routes** is a field that contains a list of IP address prefixes that were withdrawn from service. An element of the list is a tuple, which consists of a Length and a Prefix variable. The length field indicates the number of bits the Prefix variable consists of.

The next three fields correspond to the advertisement of reachable destinations by the peer. **Total Path Attribute Length** is a 2-octet value indicating the length of the **Path Attributes** field, similarly as for the previous two fields. The Path Attributes field is a list consisting of triples. Each element describes some information about the destinations that are saved in the last field.

Network Layer Reachability Information, or NLRI, is a field that contains a list of destination IP address prefixes, advertised by the peer. The format of the list and the format of elements are the same as for Withdrawn Routes.

Chapter 3

Longest Prefix Matching Algorithms

Packet forwarding is a primary function of routers. The idea of packet forwarding was already introduced in Section 2.3 and the crucial mechanism is described in this chapter. First, the problem of searching for the longest matching prefixes is described. After that, existing algorithms and structures are introduced. In this work, two of these algorithms (see Section 3.2 and Section 3.3.3), were used as subjects of experiments and studies. Therefore, these algorithms are described in more detail.

The problem of Longest Prefix Matching (LPM) is the problem of identifying the longest prefix among all prefixes that match the inspected string[25]. Consider a packet with a destination IP address with an initial byte: 101001. According to Table 3.1, the IP address matches the prefixes P_4, P_6, and P_7. However, the result of LPM will only be the P_7 prefix, because this is the longest of all the prefixes in the table. In this case, the table is a simple implementation of a forwarding table. Each prefix could then be associated with the next hop information (i.e., an output port or Data Link address)[9]. Here, the table contains only nine prefixes, but the number of entries can differ. For core routers, the number can easily pass 1 million prefixes[17]. That is a huge complication because in this case, the sequential search would be exhaustive and slow. In this case, multiple LPM algorithms were designed. In the thesis, the main focus is on algorithms that use a tree as a base structure.

3.1 Naive Algorithm

One of the simplest approaches is a linear search. For an IP address of a packet, the algorithm will try to match each of the prefixes with the IP address. Each time the longer prefix is found, it is stored in some variable. The algorithm ends when all prefixes have been tested. The saved prefix is the longest prefix found. This approach is easy and efficient for a small number of prefixes. The time complexity $O(N)$ of this algorithm is bound by N, which represents the number of entries in the prefix table.[25]

The linear search does not consider any ordering of prefixes, which could be exhaustive with regard to power and time. The following algorithms use some ordering, which is implicitly made using a special data structure.

Rule	Prefix
P_1	0*
P_2	00001*
P_3	001*
P_4	1*
P_5	1000*
P_6	101*
P_7	1010*
P_8	1011*
P_9	111*

Table 3.1: The prefix table used from [25].

3.2 Binary Trie

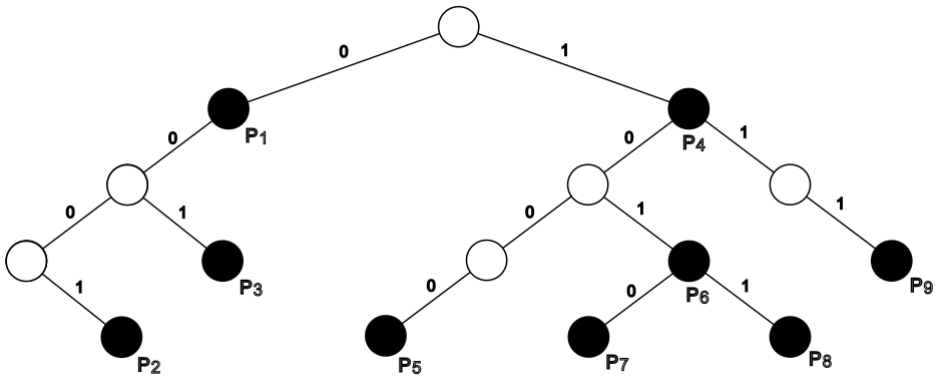

Figure 3.1: The representation of Table 3.1 in the form of a Binary Trie. The black nodes are *prefix* nodes, the white nodes are *place holder* nodes[25].

The binary trie uses a tree-like structure to represent a group of prefixes. Each node represents a prefix[24]. The depth of the trie can be the same as the maximum length of an IP address prefix, which is 32 for IPv4 and 128 for IPv6. Figure 3.1 shows the representation of Table 3.1 as a binary trie.

The structure of a trie is a simple binary tree. Nodes are of two kinds; prefix nodes or placeholders (also empty nodes)[24]. The binary trie is efficient because it provides an implicit ordering of the prefixes. The search starts at the root of the trie, which represents the empty prefix $*$. Then, any node could continue in two branches (left or right) connecting the parent node with its children. They represent new prefixes created from the prefix of their parent node, appending to the prefix value 0 or 1[24], see Figure 3.1. For example, the prefix P_3 with the value $001*$ is represented by the route from the root node of the trie, following two branches left with the value 0, and finally one branch right with the value 1. The most specific prefixes are located at the lowest levels [25].

The **search** for the longest prefix is performed by traversing the trie, dictated by the values of bits in a provided destination IP address (the IP address must first be translated into

binary format), which is then read from the MSB. If the search process arrives at a node that does not continue with any following branch or if the value in the IP address matches the branch value that does not exist in the trie, there can occur two scenarios. If the final node is a prefix node, the longest prefix has been found. If the final node is an empty node, with backtracking, the last seen prefix node is the resulting prefix.

Binary trie allows one to perform simple update operations. If a new prefix needs to be inserted, the operation is performed in two phases; first, the ordinary search for the new prefix is performed. As soon as the search ends in a node with no further branch to proceed, a new branch with the necessary nodes is created in phase two[25].

For deletion of the prefix, the process is similar to the first part of prefix insertion. First, the prefix must be found. As soon as the corresponding node is found, the prefix is removed from the node, and the node is marked empty. Now, two scenarios can occur; If the node is a leaf node (without any children), it is deleted from the trie. Subsequently, this control should be performed on all nodes before this node to avoid empty branches in the trie[25].

The time complexity is equal to $O(x)$, where x is the number of bits in the longest prefix saved in the trie. The worst case is a search for a prefix that has the same length as the address, 32 nodes for IPv4 and 128 nodes for IPv6. The same number of nodes could be inserted or deleted and thus the corresponding time complexity. Memory consumption depends on the number of prefixes, multiplied by the probability that all of them could have the maximum prefix length[25].

Binary trie allows for an easy implementation of the IP lookup algorithm. Strongness, but also weakness, is seen in processing only one bit at a time, as mentioned in [9]. For updates, the one-bit approach is efficient because the processing of nodes is more precise. But this could also cause the creation of long one-way branches. The search process then passes through a long sequence of nodes, without decision-making. This situation is undesirable. Compression of paths is a mechanism to reduce the amount of memory used and the processing time. The idea is to compress long branches into a single node (or dedicated structure) and to provide a way to bypass the long sequence of nodes with only one branch[25]. This is also one of the optimizations that can be seen in a group of algorithms known as Multibit Trie algorithms.

3.3 Multibit Trie Algorithms

A multibit trie is based on the binary trie algorithm. As described in [25], an edge in this type of trie does not correspond to a single examined bit in a prefix, but to multiple bits at the same time. The number of bits that can be examined in one step is called *stride length* (SL). A multibit trie is a structure that provides nodes, each representing a group of bits, that are matched together[24]. Each multibit trie node has 2^{SL} children. If all nodes at the same level of a multibit trie have the same SL, it is called the *fixed* stride. Otherwise, it is called a *variable* stride[25].

Before showing examples of algorithms, the concept of multibit trie algorithms needs to be introduced. Based on Literature [25], an IP prefix can be expressed as an equivalent set of prefixes after a series of transformations. There are multiple transformations available, but in this work we will describe only one of them.

Prefix Expansion If there is a prefix that is said to be transformed by expansion, it is converted into multiple longer and more specific prefixes that cover the same range of addresses. In addition, if we do this for some group of prefixes with different lengths, after the transformation, we get a set of prefixes with less different lengths[25]. To show the idea in the example, consider a multibit trie M with fixed stride $SL = 3$. An expansion can be done when the length of some prefix is not multiple of SL and the prefix must be saved in M (for example, prefix P_1 from Table 3.1). First, the prefix must be expanded to 3-bit sequences: $000*, 001*, 010*, 011*$. After that, the sequences need to be checked to see if some of them correspond to different and more specific prefixes. For example, $001*$ corresponds to a prefix P_6. Next, the sequence $000*$ is associated with prefix P_1. But it is also part of longer prefix P_2 ($00001*$), which will be specified at the lower level of the trie. This information also needs to be stored. The resulting expanded multibit trie, corresponding to Table 3.1, can be as follows, see Figure 3.2.

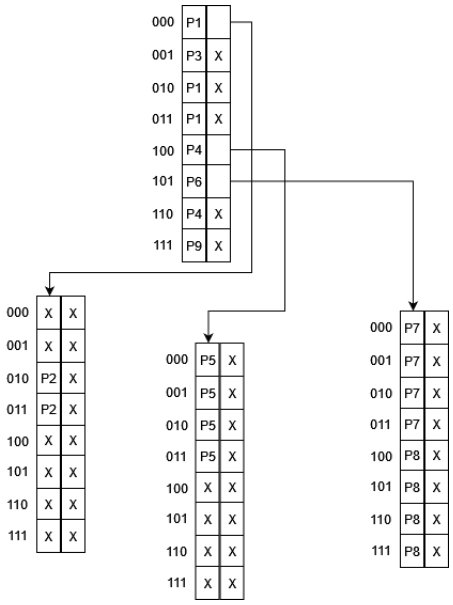

Figure 3.2: The scheme of a multibit trie with $SL = 3$, after an expansion transformation of Table 3.1.

Disjoint Prefix

When the prefix expansion generates new prefixes, a disjoint prefix transformation uses a different principle. The set of a disjoint group of prefixes means that no two prefixes of the group overlap. An overlap of two prefixes occurs when one interval of addresses (defined by one of the prefixes) contains another interval of addresses (defined by the other prefix). This is also the reason why address lookup can match multiple prefixes at once (as demonstrated in the introductory part of Section 3.3)[25]. In this sense, disjoint prefix transformation processes the prefixes in the following steps; Construct a binary trie from the set of prefixes. For all nodes that have only one child, a new leaf node will be created. These new nodes represent new prefix nodes. In this sense, these nodes inherit the forwarding information from the closest ancestor marked as a prefix. Lastly, the prefixes of the internal nodes are removed.[25]. The principle in which this transformation works is also known as *leaf pushing*[10].

3.3.1 Multibit Trie

As was described in the introductory part of Section 3.3, an SL value can define how the multibit trie is decomposed. For tries with fixed stride, the algorithm is called *fixed stride multibit trie*. In contrast, tries with variable stride are called *variable stride multibit tries*[25]. In each case, these structures store prefixes that might have to be expanded to the SL or SLs supported by the structure.

One of the examples can be seen in [9] which was also a template for the scheme in Figure 3.2. This is a scheme of a fixed stride multibit trie with $SL = 3$. Each two-column table in this scheme corresponds to one node. It is obvious that each node needs to store a list of all 3-bit combinations (8 entries in total); each of these entries must also store two pointers. A pointer to the corresponding prefix and a pointer on nodes of a lower level of the multibit trie. The scheme can also be used as a template for the possible implementation of fixed stride multibit trie. Entries that do not correspond to any more specific prefixes or subtrees are saved as empty entries (see the left node of the second level, which contains six empty entries). Based on the shown example, it is clear that expansion transformation combined with multibit trie may generate new nodes or node structures, which can in the end store no information or redundant information. In this example, a small value of SL was used, and the resulting inefficient space use can be omitted. However, the use of larger SL values can require the use of larger trie nodes with a higher memory overhead[10]. In this sense, another approach and solutions were designed.

3.3.2 Lulea

A Lulea algorithm is a fixed stride multibit trie algorithm that uses modified node structures to minimize a used space. In this sense, it provides the ability to use larger SL values. As discussed in Section 3.3.1, simple multibit trie with a large SL value will likely have oversized nodes that waste a lot of space[25]. For Lulea, this space is compressed by implementing one rule; Each node with a list of entries (we can consider the scheme in Figure 4.2) has to contain either prefix information (for example, next hop information) or a pointer to the child node. This affects the overall structure of the trie. In the structure, there cannot exist nodes that are both prefix nodes, but not leaf nodes. The information from internal nodes of a trie need to be *pushed down* into leaf nodes[10]. This transformation was already discussed in Section 3.3. Furthermore, for more space-saving requirements, instead of using a list of entries, Lulea uses bitmaps. A similar approach was used in the design of another LPM algorithm. In this way, the mechanism of bitmaps is described in the following part, in Section 3.3.3. The only difference is in the insertion and deletion processes. While Lulea has to rebuild the entire structure, a Tree Bitmap algorithm provides more efficacy[25].

3.3.3 Tree Bitmap

A Tree Bitmap algorithm (TBM), as mentioned in [24], is known as the most effective multibit trie algorithm. It is a fixed stride multibit trie algorithm that allows fast searches and provides storage efficiency comparable to the Lulea algorithm[25]. It represents a set of prefixes by mapping TBM nodes on a binary trie. Each of these nodes covers a trie's subtree of maximum depth equal to the SL. Each of these nodes covers part so that the nodes do not overlap; see Figure 3.3. The idea behind TBM lies in the use of *bitmaps*. As already mentioned in Section 3.3, the structure of nodes in the most simple multibit trie

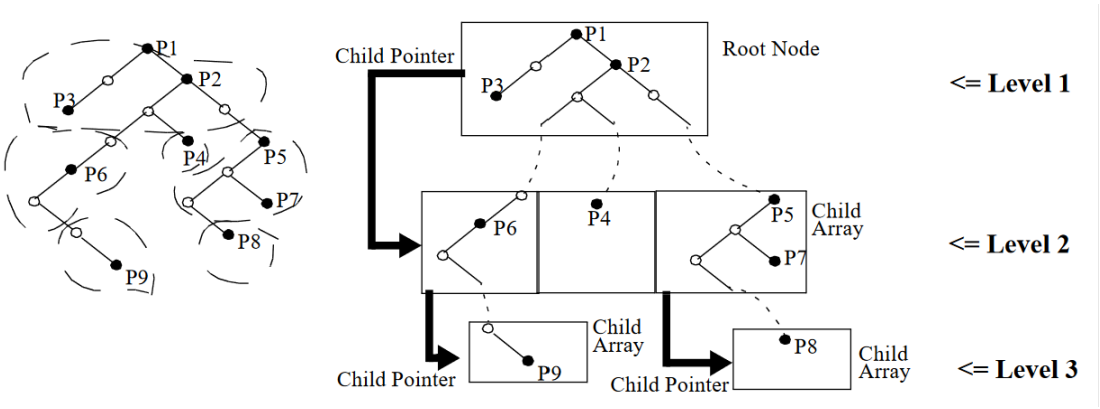

Figure 3.3: The illustration of mapping TBM nodes onto binary trie, $SL = 3$[9].

can contain redundancy, leading to inefficient use of memory (empty entries, redundant information in one node, etc.)[9]. In TBM, information about the node with a specific SL and the following child nodes is encoded in two bitmaps. Each bitmap then serves as a way to calculate the offset in a list of prefixes or child nodes. In addition, prefixes and children's information can be saved in continuous data blocks at a different location. This leads to more compact structures and a simpler structure for the TBM nodes[9][24].

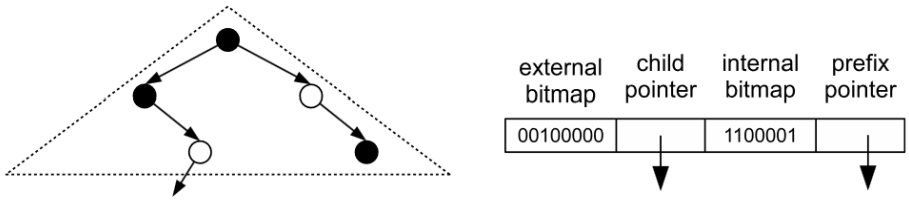

Figure 3.4: Example of the encoded root TBM node [24].

The structure of a TBM node can be seen in Figure 3.4; an **external bitmap** contains bits, where each of them represents the following child node, indexed from the left-most child corresponding to MSB to the right-most child corresponding to LSB[24]. The next part of the node structure is a pointer to the list of children (see Figure 3.3). The value of the corresponding bit of the external bitmap expresses if the child node exists. Thus, the node is saved in the child list (1) or not (0). The order of the nodes in the list must correspond to the indexing in the bitmap. Access to the child node is done using pointer arithmetic[9].

Next, there is an **internal bitmap**. Here is a piece of information about the prefixes that correspond to nodes inside the TBM node. As can be seen in Figure 3.4, a TBM node covers a subtree of a simple binary trie, see Figure 3.1. Each node inside of this subtree could correspond to a prefix. This information is then saved inside the internal bitmap. The number of bits in the bitmap is equal to $2^{SL} - 1$. The nodes of the subtree are indexed according to the breadth first order[24]. Similarly to an external bitmap, the value of a bit in the internal bitmap expresses whether the prefix exists or not. Accessing the prefix information is done using pointer arithmetic. Next to this bitmap, a pointer to a list of prefixes is provided.

As discussed in the previous paragraphs and mentioned in [24], the final structure of a TBM node contains only two pointers and two bitmaps. The length of the bitmaps depends on the SL of the implemented TBM. How the useful data are stored makes it possible to access them using fast pointer arithmetic. Encoding subtrees of the underlying trie into bitmaps allows the implementation of LPM in hardware, which has better performance. Finally, a fixed structure of the nodes provides easier and faster updates. However, it should be mentioned that the number of matching steps and the memory used to store the structure depend greatly on the value of SL. There could be situations when the underlying trie is sparse and the use of TBM with higher SL value could cause high memory overhead. On the other hand, a higher SL value provides a lower number of matching steps. The factor that causes this problem is the fixed shape of a TBM node, which always represents a full subtree of depth $2^{SL} - 1$ (see previous paragraph). In the next algorithm, a new way is described, which should help to overcome the problem.

3.3.4 Shape Shifting Trie

The Shape Shifting Trie (SST) is an algorithm that also encodes nodes as TBM. Additionally, SST encodes the shape of the underlying binary trie. Paper [30] mentions that with this approach, SST can solve the problem (see Section 3.3.3) of high memory overhead if the underlying binary trie is spare and of inefficient shape[24]. SST does not use the SL variable to determine how many nodes of the underlying trie are represented by one node, but uses parameter K. This parameter describes the maximum number of nodes in the underlying binary trie, which could possibly be represented by one SST node.

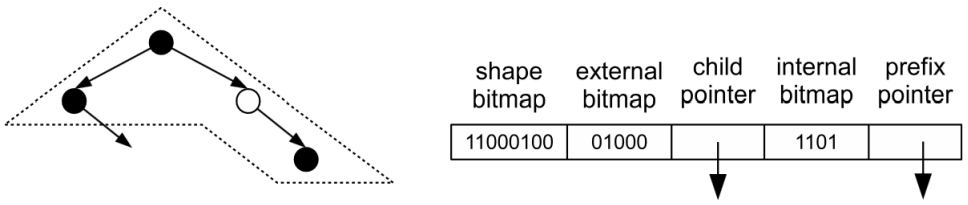

Figure 3.5: The encoding of an exemplary subtree of a binary trie into SST node for $K = 4$[24].

The data structure of the SST node is shown in Figure 3.5. The purpose of the internal and external bitmaps and pointers is the same as that of Section 3.3.3. The difference is in the number of bits stored in the bitmaps. For the internal bitmap, the number of bits is K, and for the external bitmap, the number of bits is $K+1$[30]. Next, the shape of the subtree trie is encoded in the so-called *shape bitmap* (see 3.5), which contains $2K$ bits. Each node of the underlying binary trie is then represented by the pair of bits, where the left bit corresponds to information if the node has left child (1) or not (0). The same situation applies to the right bit and the right child. These bits are then listed in Breadth-first search order[30].

The SST algorithm utilizes nodes that use a small amount of memory to store information. To create these nodes, it adapts their shape based on the underlying trie nodes. However, construction and update are computationally complex. Furthermore, shape bitmap decoding has a negative effect on search performance. In this way, SST is not an effective LPM algorithm for prefix matching in core routers. However, the described approach could be used as an inspiration for other algorithms[24].

Chapter 4

SandRouter

SandRouter (the name inspired by the term in the field of computer security, Sandbox[1]) was designed as a support framework for the development of new packet classification algorithms. The framework should help to clarify how the structure of the FIB table changes when processing network traffic. The first part describes why SandRouter was designed and shows it from a high level. In the second part, the general architecture and elementary modules of the framework are described. The third section describes which statistics are tracked, why they could be important, and how they could be visualized.

4.1 High-Level Design

Packet classification algorithms are unique in their use of certain data structures. From the nature of these algorithms and implementations mentioned in Chapter 3, the structures can be seen as hierarchical. This allows us to identify the properties common to these structures. Furthermore, we can assess both qualitative and quantitative characteristics, which can then be used to compare various implementations of FIB tables. At this point, we can state that we are not interested in performance metrics, such as search, storage, deletion speeds, and related aspects. The primary focus is on examining the dynamic changes in data structure.

It is convenient for further design to introduce how this framework is planned to be used to benchmark implementations of FIB tables. Let us assume that there is a user who has implemented a new classification algorithm of a FIB table (named *model*). He would like to test the model by putting it into a router (or some sort of simulation) to see how the traffic affects the shape of the overall FIB table data structure. He would like to perform the testing as truthfully as possible and to receive an overview of how the FIB table structure changed. SandRouter as a benchmark framework will take the user's model and declare it as the new FIB table. As such, the model will be used in a simulation of a real router. SandRouter will monitor the model during this simulation and collect information about how it changes data structures. In the end, it will provide results in the form of a statistical output.

4.1.1 Design Considerations

The first step in designing a framework was to do a research on existing frameworks for inspiration. Some of them are described in this part. One of the ideas is to allow a user

[1] `https://cs.wikipedia.org/wiki/Sandbox`

to integrate the model into a real operating router. The router has to be equipped with extra functionalities; it should provide the possibility of changing the implementation of its FIB table. The change and subsequent deployment should be simple. Next, it should be able to monitor the model. Finally, the router should be able to provide results. This approach would also enable running benchmarks in a real environment, without the need to simulate the whole router. But the requirements mentioned beforehand are difficult to meet. The software on routers is usually proprietary and manufacturers explicitly prohibit any modifications. Another approach could be to use software or a module that acts as a virtual router. This implies a solution of modular routers with configurable architecture[21]. Still, this solution does not provide an extension to change the FIB table algorithm. Another existing solution could be a dynamic IP routing daemon, for example, BIRD[2]. However, it's configuration is complicated and it has a FIB structure scheme with no support for changes.

Another factor that has an impact on the SandRouter's design is the source of traffic for simulations. One simulation should run in a tolerable amount of time. If the framework with a model was directly connected to some network, the passing traffic might be insufficient. In this case, there could be a router, or a daemon, which is connected to frequent network traffic. For example, core routers, where the traffic is the busiest. However, it could be expensive to use the real core router. Perhaps there is no need for a real-time data stream at all. It would be sufficient to have a collection of saved traffic, which will then be replayed by SandRouter. Furthermore, a user can select the data that will be used for a benchmark simulation.

Using a real router or some virtual module was not the right way. But for the thesis, the behavior of SandRouter does not have to exactly copy the behavior of the real router. In Section 2.3 it was mentioned that a FIB table is built based on the information received from a RIB table. This RIB table analyzes messages received from routing protocols, evaluates the relevance of information from these messages (recognizes new prefixes to be stored or invalid prefixes to be removed) and eventually pushes them to the FIB table[8]. Then, the FIB table stores the information in its structures. This is a simplified illustration of communication between a RIB table and a FIB table inside a router. This is also a basic concept that should be enough for SandRouter's design. The described system can be integrated as a whole into the SandRouter framework. The implementation of the FIB table will be the only exchangeable part. This could lead to a modular design of user implementations. The functionality of a RIB table, monitoring, and processing of traffic will be managed by SandRouter.

4.2 Framework Architecture

SandRouter is designed as a standalone application, which will monitor and interact with the FIB table model provided, query for information about the actual state of the model, and collect data for the output statistics. The system consists of four parts: RIB module, FIB module, Statistics Collector and Manager. Together, they represent a simplified router. Figure 4.1 shows a simple scheme of the entire system. Each part is described in more detail. After that, the concept of simulations is described.

[2] `https://bird.network.cz/?index`

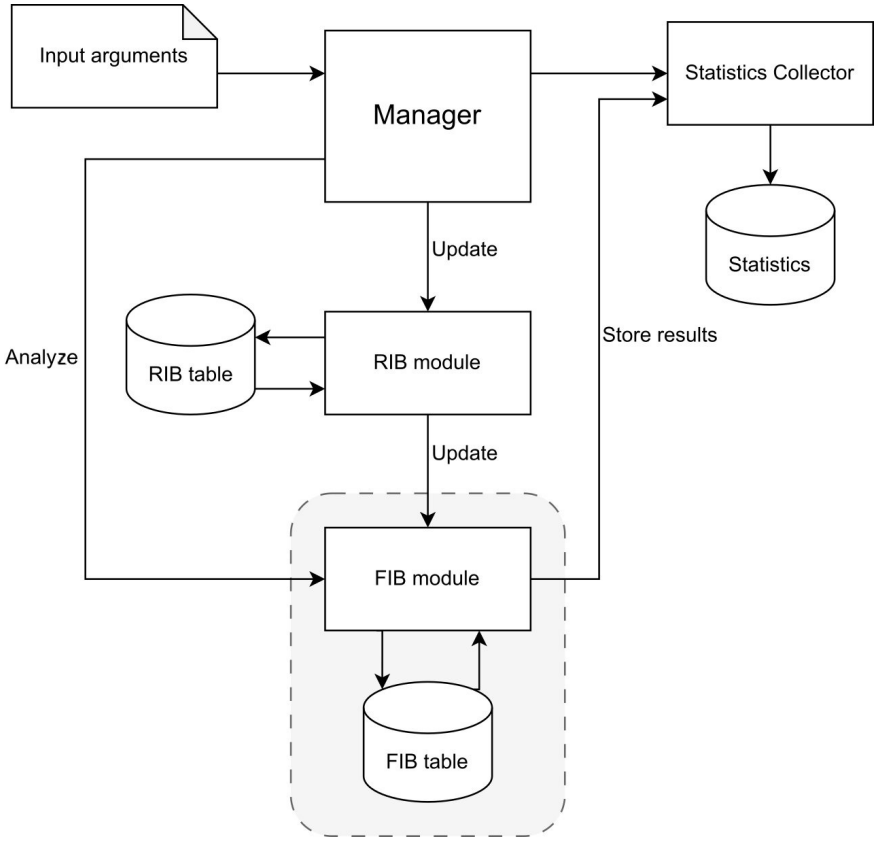

Figure 4.1: The scheme of the SandRouter framework. Parts inside a dashed rectangle correspond to the part of the system, which could be changed by a user.

4.2.1 Manager

The Manager is the main module that controls the whole system. It can be pictured as the main process inside a router. It controls and orchestrates simulations. Before starting a simulation, it first preprocesses the *input arguments*. The input arguments consist of the configuration parameters and BGP data. The data consists of two files; *snapshot* file, which contains the base structure of the RIB table and the FIB table. This information is needed before the simulation is triggered. The second, *updates* file, contains BGP update messages. During simulation, the Manager sends update messages to the RIB module, where they are processed. Between sending updates to the RIB module, the Manager periodically sends a trigger message to the FIB module to analyze its FIB table. The results are then handed over to the Statistics Collector. A dedicated input parameter controls the frequency of these analyses. At the end of the simulation, the Manager notices the Statistics Collector to generate results.

4.2.2 RIB Module

The RIB module represents a simplified Routing Information Base. It contains the RIB table, which stores information of all seen prefixes and their history. The module analyzes which updates are pushed to the FIB module. These decisions are made based on the content of a RIB table.

4.2.3 FIB Module

This module contains the implementation of one of the LPM algorithms, which implements the FIB table used in the simulation. Based on the update received from the RIB module, the FIB table is updated. During the simulation, the Manager will ask the FIB module to analyze the actual state of its FIB table. The response will then be collected in the Statistics Collector.

In addition to the implementation of FIB functionalities, this module also provides the ability to change the LPM algorithm. If a user would like to test his implementation of a different LPM algorithm following simple interface requirements, he can change the LPM algorithm used in the FIB module. After that, with the use of a special input parameter, the SandRouter will simulate the provided user model.

4.2.4 Statistics Collector

This module will collect information about the FIB table received from the FIB module. During the simulation, the Statistics Collector is called by the FIB module and is provided with stored information. After the simulation, the Collector is notified by the Manager to generate results in the form of graphs; see Section 4.3.

4.2.5 Simulation

As part of the whole SandRouters design, simulation is where all modules co-work together. The process is divided into several steps; see Figure 4.2. The first two steps correspond to the preprocessing and preparation of the data structures, especially the FIB module and the RIB module. These two parts contain the important RIB table and the FIB table, based on which decisions are made. The FIB table is the studied object, and its default state is crucial for further analysis.

The next part represents the main process loop, which ends after the whole content of the update file is processed. Each update has the following format: A/W [prefix]. The first character represents an operation, which should be done with the following prefix. *A* stands for *Append* and *W* stands for *Withdraw*. Based on that, the Manager commands the RIB module to save or delete the prefix in the RIB table. If the prefix is new (there is no information about it in the RIB table) or if the last information about the prefix is going to be removed, the information will be pushed to the FIB module. The information is then saved to or removed from the FIB table. Each of these actions is properly recorded and stored in the Statistics Collector to provide a dynamic statistic.

One of the input arguments is a sequence of updates, after which is done a snapshot of the actual state of the FIB table. The snapshot is then stored in Statistics Collector. As all the updates are processed, the Manager commands the Collector to generate final statistics and store them in the provided location. The simulation is then terminated.

4.3 Designed Statistics

In Chapter 3, some of the LPM algorithms are mentioned. The impact of hardware implementation of these algorithms was also mentioned. The way the algorithms work could cause complications in memory access[24]. Therefore, there is a place to look at how classification algorithms work with data structures. In the context of IP prefix classification,

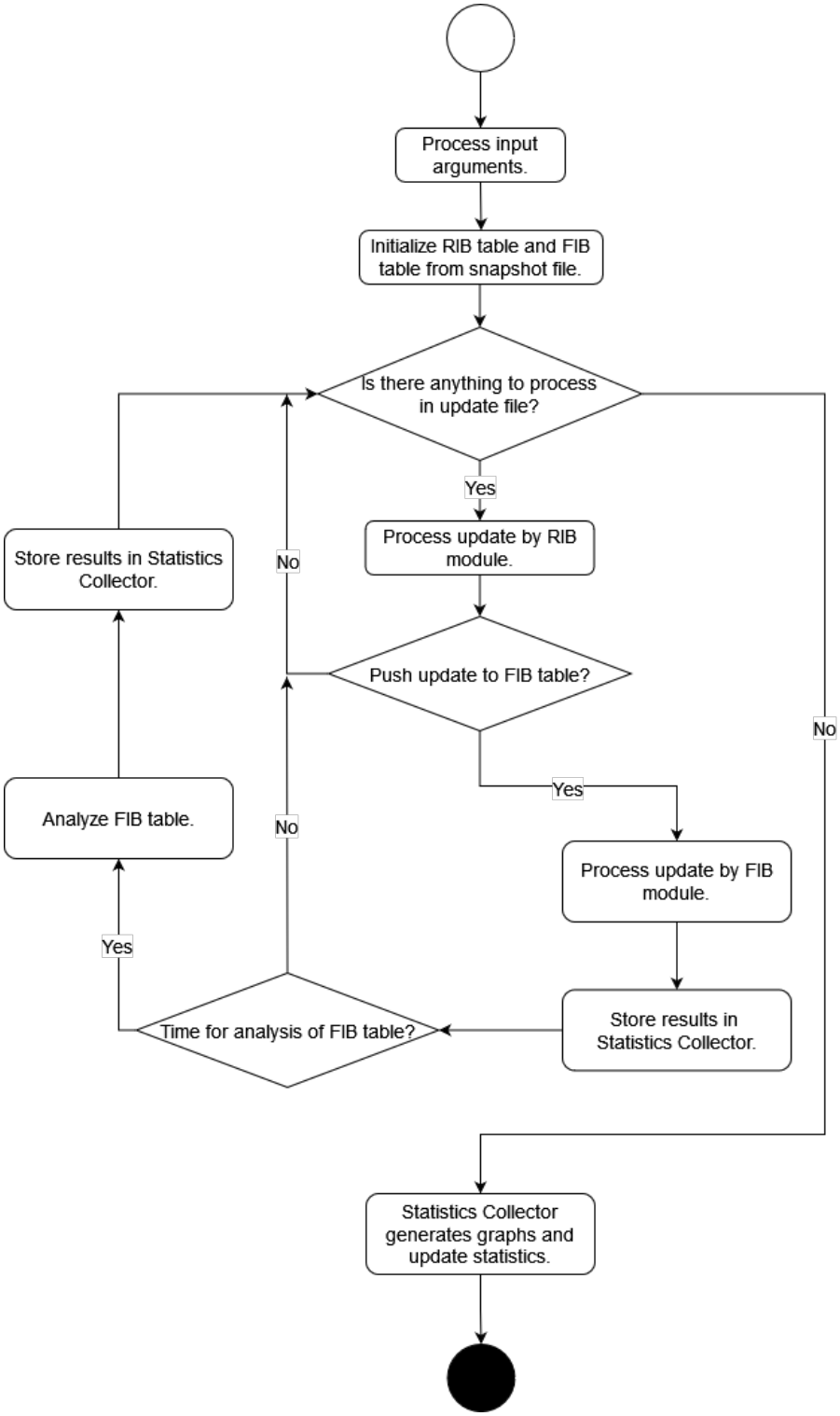

Figure 4.2: Scheme of a simulation, described in steps. The white circle represents start of a new simulation. The black circle represents the end.

the basic structure of the FIB table is a Binary Trie, which is nothing more than a simple binary tree. Therefore, it could be helpful to consider some of its properties.

In real routers, the FIB table is built as a simplified version of the RIB table. If we consider the FIB table to be a simple Binary Trie (see Section 3.1), we can describe the shape of this structure. For example; the maximum depth of the trie, the the number of nodes on each level, the maximum number of children appearing on each level, etc. Some of the properties may seem unnecessary in the context of Binary Trie. However, SandRouter is not designed to test only LPM implementations using simple Binary Trie. If we look at more complicated algorithms, such as Tree Bitmap (see Section 3.3.3), the number of children for one node could grow exponentially, depending on the SL (stride length) used. This implies the need to allocate more memory to save the structure of one TBM node. Furthermore, this situation could be different at each level.

4.3.1 Structural Statistic

This group of properties was designed to show how the shape of the FIB table changes through a simulation. All of the following properties are collected by snapshots of a FIB table. These snapshots are made in the step *Analyze FIB table*, see Figure 4.2. The results are then plotted in a box and whisker plot graph for each level, which appears in a FIB table. For each level, every of the following properties is provided:

- **The number of nodes** This information reflects how dense the structure is at the level. A node at some level represents the corresponding part of some prefix (with a value of 0 or 1 at the corresponding index).
- **Number of prefixes** Not every prefix node is also a leaf node. If the node is on the searched path, it serves as a fallback solution if a more specific prefix is not found.
- **Maximum number of children** This property provides the highest number of children that appeared in one of the nodes, on the corresponding level. This is a good indicator of the maximum space that needs to be potentially allocated to one node.
- **Maximum number of prefixes** Similarly to the previous property, but in the context of allocation of space for prefixes in one node.

4.3.2 Dynamic Statistic

The FIB module stores the dynamic changes in a FIB table and reports them to the Statistics Collector. This is performed in the step *Process update by FIB module*, see Figure 4.2. These updates can be of two kinds; they could add a new prefix or delete an old one. The actions that follow each of these two differ. With that, two statistical groups were designed. One represents the dynamic of adding new information to the data structure, while the other represents the dynamic of removing some information. Both groups have the same statistical properties. The only difference is in context.

- **Number of updates** This property stores the number of updates that were performed throughout the simulation.
- **Level access** For each level, the number of accesses by updates is stored. From this value is then calculated the percentile of updates that accessed the level.

- **Created/Removed nodes** The number of node structures created or removed throughout the simulation. It is the summary value for the whole FIB table.
- **Min and Max depth** These two properties correspond to the highest level and the lowest level accessed throughout the simulation.

The last property shown in Dynamic Properties is **Minimum and Maximum the number of nodes**. This is part of the so-called *Total number of nodes*. This group was created to show the overall state of each level, including both types of updates. The number of nodes on each level is already provided in the Structural Properties (see Section 4.3.1), visualized in the graph. The exact values are also provided here to show the relationship of properties in detail.

The main focus of the study are the dynamic changes in the FIB tables. This means that Dynamic Properties are a more important output. The structural properties help to show how much space the structure can take up. They can also provide information on the worst case in the context of space consumption of each node. However, the dynamic behavior can show the dynamic of the whole level. In this way, other interesting factors can be shown.

Chapter 5

Implementation and Validation

This chapter is dedicated to the description of the implementation of the SandRouter framework. First, the technologies used are described, followed by a part focusing on the implementation of each module introduced in Section 4.2. After that, there is a section with the description of validation of SandRouter. The support of two IP protocol versions and the implementation of the two selected LPM algorithms are also described. After implementation, the description of developed scripts is introduced. Finally, Section 5.5 describes how to implement a user-defined FIB model and what methods should be provided.

5.1 Used Technologies

For the implementation of a framework was used the high-level interpreted language Python version 3[1] and some of Python libraries:

- argparse[2]
- Path from pathlib[3]
- matplotlib[4]
- ipaddress[5]
- collections[6]

As a development environment, system Windows 11 with installed Windows Subsystem for Linux (WSL) was used.

5.2 Implementation

The execution file, *SandRouter.py*, contains the entry point for the execution of a simulation. The first step is to parse the input arguments. These arguments provide paths to source files and configuration parameters, specifying the execution of a simulation.

[1] https://www.python.org/download/releases/3.0/
[2] https://docs.python.org/3/library/argparse.html
[3] https://docs.python.org/3/library/pathlib.html
[4] https://matplotlib.org/
[5] https://docs.python.org/3/library/ipaddress.html
[6] https://docs.python.org/3/library/collections.html

The overview of the parameters is as follows:

- **model** - model selection
- **period** - period of updates between analyzes
- **version** - version of the IP protocol
- **snapshot** - path to the file containing snapshot structure for RIB and FIB modules
- **updates** - path to the file containing updates for simulation
- **destination** - path to the destination folder, where results will be stored

For more information see the help message of *SandRouter.py*

The second step is the initialization of the RIB table and the FIB table using a method provided by the Manager module (see Section 5.2.1). This method returns two objects corresponding to the RIB object and the FIB object. The FIB object can be of three types, based on the selected model for the FIB table (for more information, see Section 5.2.3). Next, the Statistics Collector module, which consists of classes *UpdatesAnalyser* and *OverallAnalyser* (see Section 5.2.4), needs to be intialized.

After initialization, the actual simulation can be run. The whole process is done in the method *Simulation(...)* implemented in the Manager module. This method takes as arguments initialized RIB and FIB objects, and objects of the Statistics Collector. After completion of the simulation, statistical results are generated. Generation is performed separately for Dynamic Statistics (by method *UpdatesAnalyser.FinishAnalyse(...)*) and structural statistics (by method *OverallAnalyser.GenerateGraphs(...)*). The results are then stored in the destination folder (see the list of input arguments in this section).

5.2.1 Manager Module

This module contains methods to prepare and run simulations. Based on the input arguments, processed after start, the methods set up multiple resources.

- `def GenerateTables(_file: str, _model: int, _type: str) ->tuple[RIB_Class, TreeBitmap|BinaryTrie|FIBModel]`

  This method initializes the RIB table and the FIB table structures. The *_file* argument contains a path to the *base.txt* file. The *_model* argument contains the selected model of the FIB table used in the simulation. The last argument, *_type*, indicates the version of IP protocol used for prefixes inside input files. As the return value, the method returns two objects. *RIB_Class*, which is a base class implementing the RIB table. And the class corresponding to one of the models (see Section 5.2.3).

- `def Simulation(_file:str, RIB:RIB_Class, FIB:TreeBitmap| BinaryTrie|FIBModel, updateAnalyser:UpdatesAnalyser, overallAnalyser:OverallAnalyser, _iter:int, _type:str)`

  This method carries out the implementation of a simulation. The input arguments are, *_file* argument containing the path to the *updates.txt* file. *RIB* argument represents RIB table and *FIB* argument represents FIB table. *updateAnalyser* and *overallAnalyser* arguments are objects of Statistics Collector for collecting statistical

data through simulation. *_iter* argument stores a period of updates, after which a FIB model will be instructed to carry out *GetStatistics(...)* method (see Section 5.5). Because this method scans the whole FIB table data structure, it can significantly slow down the process if executed too often. If the *_iter* value is higher (in the hundreds), the simulation will run faster at the expense of lower accuracy of Structural Statistics results. This parameter is used to control how often is the overall analysis made (see Section 5.5 with a description of *GetStatistics(...)* method). This method executes a walk through the whole FIB table data structure and collects statistical data, which are then handed to Statistical Collector. This method can be expensive in respect to time, because the structure of the FIB table can contain more then a milion nodes (as an example, see Figure 6.1). The final argument, *_type*, indicates the type of IP protocol of prefixes inside the *updates.txt*.

Algorithm 1 Pseudocode of the main cycle in method Simulation(...)

```
1:  for update in updates.txt do
2:      Parse update into action and prefix
3:      if _type equals „ipv4“ then
4:          addr ← AddressIPv4(prefix)
5:      else
6:          addr ← AddressIPv6(prefix)
7:      end if
8:      if action equals 'A' then          ▷ 'A' stands for Announce prefix, see Section 2.4.1
9:          val ← RIB.SavePrefix(addr)
10:         if val is True then
11:             result ← FIB.SavePrefix(addr)
12:             updateAnalyser.SaveAction(result)
13:             Increment iter
14:         end if
15:     end if
16:     if action equals 'W' then          ▷ 'W' stands for Withdraw prefix, see Section 2.4.1
17:         val ← DeletePrefix(RIB, addr)
18:         if val is True then
19:             result ← FIB.DeletePrefix(addr)
20:             updateAnalyser.DeleteAction(result)
21:             Increment iter
22:         end if
23:     end if
24:     if iter > _iter then
25:         Reset iter to 0
26:         result ← FIB.GetStatistics()
27:         overallAnalyser(result)
28:     end if
29: end for
```

Because *Simulation(...)* is the most important method of the SandRouter implementation, the pseudocode of the main simulation loop is provided. The names of the input arguments are the same as in the previous paragraph. The algorithm iterates through the list of updates provided by *updates.txt* file. From each update is extracted the type

of the action and the prefix (see the format of updates explained in Section 4.2.5). Based on the action is decided, if the prefix is going to be stored in the RIB table, or removed from it. In both cases, if the RIB table return value equals $True$ (see statements at Lines 10 and 18), the action is further propagated into the FIB table. At this point, the method of the FIB module returns statistical information of the action. That is stored in *updateAnalyser* class and *iter* variable is increased. After the action, the variable is compared with the argument *_iter*. If the statement is true, the *iter* variable is going to be reset, and the *overallAnalyser* stores the statistical data.

5.2.2 RIB Module

This module contains an implementation of class *RIB_Class*. This object is a simplified implementation of the Routing Information Base inside a real router. As mentioned in Section 2.3, the RIB table contains paths to various destinations. In addition, the RIB table needs to store the properties of these paths, which are needed in routing algorithms.

In the SandRouter framework, the *RIB_Class* implements a base structure of the RIB table, which is implemented as a Binary Trie (see Section 3.2). Each *RIB_Class* object is a node in the RIB table structure. The attribute *prefixCounter* (see Listing 5.1) replaces the list of alternative paths. As visible in Algorithm Lines 8 and 16, the type of update can save a prefix or delete it. Without any further context of these updates, it is seen that each appearance of an update with the prefix can increment the number of alternatives, or decrement it. The return value from both methods (see Algorithm Lines 9 and 17) indicates if the prefix appeared for the first time, in this way, needs to be stored in both tables. Or, the prefix was deleted from the RIB table, indicated by the value of the *prefixCounter* being 0, and needs to be removed from the FIB table as well. For more details, see the source code file *RIB.py*.

```
class RIB_Class:
"""
Attributes:
    prefixCounter (int): number of existing alternatives
    left (RIB_Class): a left child of a parent node
    right (RIB_Class): a right child of a parent node
    mask (int): an overview of how deep in the RIB structure the node is
"""
```

Listing 5.1: Attributes of a class Base_Class

5.2.3 FIB Module

This module contains three implementations of FIB table models. Each model corresponds to a class that can be selected as an implementation of the FIB table in the actual simulation. The model can be selected by the dedicated input argument (see the list of input arguments at the beginning of this section).

Binary Trie

This model corresponds to the Binary Trie algorithm, introduced in Section 3.2. This algorithm was selected as one of the most basic LPM algorithms because of its simple imple-

mentation. Based on this, it was used to test and validate the functionality of SandRouter. The model is represented by *class BinaryTrie* (see source code file *FIB/BinaryTrie.py*).

Tree Bitmap

This algorithm was selected to represent Multibit trie algorithms. The implementation of this model was made based on the introduction in Section 3.3.3. Tree Bitmap was implemented with $SL = 3$, where the parameter was selected for demonstration and experimental purposes. This model is represented by *class TreeBitmap* (see source code file *FIB/TreeBitmap.py*).

User Model

This model is prepared as a template for customized implementations of the FIB table that are subjected to benchmark testing. SandRouter supports the ability to select this model during execution using a dedicated input argument (see the list of input arguments at the start of Section 5.2). More information is provided in Section 5.5. This model is represented by *class UserModel* (see source code file *FIB/UserModel.py*).

5.2.4 Statistics Collector

This module contains implementation of collectors for statistical data. Following is a short description of both classes.

OverallAnalyser is a class that stores and processes statistical data for Structural Statistics. The statistical data for the structure of the FIB table are provided by the method *GetStatistics(...)*. The data is gathered in levels (see the description in Section 5.5). This information needs to be preserved because the statistics are provided for each level separately. Because of this, the *class Level* was developed to store all information corresponding to each level. The description of the level object is shown in Listing 5.3. All these objects are stored in an array attribute, described in Listing 5.2.

```
class OverallAnalyser:
"""
Attributes:
    Data (array of objects of class Level): an array containing
    objects representing each level of the whole FIB table
    data structure
"""
```

Listing 5.2: The description of attributes of a class OverallAnalyser.

```
class Level:
"""
Attributes:
    nodesNumber (int array): array of the total number of~nodes,
    from each iteration
    nodesWithPrefixNumber (int array): array of the total number
    of~prefix nodes, from each iteration
    leaves (int array): array of~the total number of leaves
    in each iteration
```

```
    maxChildren (int array): array of maximum values of children
    seen in each iteration
    maxPrefixes (int array): array of maximum values of prefixes
    seen in each iteration
    level (int): to which level the object corresponds to
"""
```

Listing 5.3: The description of attributes of a class Level.

UpdateAnalyser is a class that stores and processes statistical data for Dynamic Statistics. Statistical data are provided by the *SavePrefix(...)* method and the *DeletePrefix(...)* method implemented by the selected FIB model. This is why the *UpdateAnalyser* contains two attributes of *AnalyseProperties* class (see Listing 5.4). Separate statistics for both types of statistical data. These statistics are stored in a specialized class, *AnalyseProperties* (see Listing 5.5).

```
class UpdatesAnalyser:
"""
Attributes:
    insert_properties (AnalyseProperties class): corresponds to
    dynamicity of insertion
    delete_properties (AnalyseProperties class): corresponds to
    dynamicity of deletion
    levels (int array): history of total numbers of nodes on each
    accessed level
"""
```

Listing 5.4: The description of attributes of a class UpdateAnalyser.

```
class AnalyseProperties:
"""
Attributes:
    action_updates_array (int array): array of history of accessing
    and modification of each level
    updates (int): the total number of updates
    depths (int array): the history of accessed and modified
    lowest depth of each update
    modifiedNodes (int): the total number of modified nodes
"""
```

Listing 5.5: The description of attributes of a class AnalyseProperties.

5.2.5 IPv4 and IPv6 Support

The SandRouter framework supports two versions of the IP protocol. For this purpose, two classes were developed, each providing functionality to manipulate IP addresses. Each class implements a specialized method *GetBitValue(...)*, which enables one to retrieve a value of a bit whose position is determined by the input argument (see Listing 5.2.5).

```
def GetBitValue(self, _mask: int) -> int:
```

5.2.6 Generation of Statistics

The Statistics Collector module provides the functionality to process and generate the resulting statistical data after the completion of a simulation. The statistics collected in *UpdatesAnalyser* class and *OverallAnalyser* class are processed by dedicated methods, implemented by each class.

```
def FinishAnalyse(self, _file: str):
```

Dynamic Statistics are generated in method *FinishAnalyse*. The input argument is a path to the file, where the resulting statistics will be stored. The whole implementation is provided in the source code file *StatisticsCollector/GraphGenerator.py*.

```
def GenerateGraphs(self, _folder: str):
```

This method generates graphs from collected statistical data, stored in attributes of *Level* class, representing each level of the structure of the FIB table. Using the *matplotlib.pyplot library* library, the data is plotted onto boxplot graphs. The resulting graphs are stored in the folder, provided as an input argument of the method. The whole implementation is provided in the source code file *StatisticsCollector/AnalyseUpdateStats.py*.

5.3 Scripts

Two scripts were developed to help preprocess the simulation data. This part is dedicated to describing these scripts.

5.3.1 *address_parser.py*

This script was developed to process a stream of prefixes and updates. It can parse and filter out updates and prefixes, based on the input arguments provided at the execution. For more information, see the help message of */scripts/address_parser.py*.

5.3.2 *SandRouter_preprocessing.sh*

SandRouter uses data originating from BGP monitors for simulations and initialization. These monitors provide databases accessible online that store communications captured by BGP collectors. These raw BGP data are provided by two types of compressed files. The first is a file that contains a snapshot of the Routing Information Base. The second is a file, which contains updates (BGP messages) that have passed the collector since the snapshot of the RIB. Typically, the first file is generated once in a few hours, and updates are collected and provided in minutes. All these files are compressed by gzip (.gz) or bzip2 (.bz2)[29].

This script provides a simplified way to generate data that are then used by SandRouter. For processing raw BGP data, it uses the *bgpdump*[7] framework and for filtration and parsing updates, it uses *address_parser.py* script. In addition, this script provides configurations to filter out a specific group of prefixes. It prepares both *base.txt* and *updates.txt*. For more information, see the help message of */scripts/SandRouter_preprocessing.sh*.

[7] https://github.com/RIPE-NCC/bgpdump

5.4 Validation

The SandRouter functionalities were tested and validated throughout development. No automatic testing system is provided. However, short simulation scenarios were designed that contain possible edge cases. The tests are located in a folder *Tests/*, which contains two other folders. Each is dedicated to one version of the IP protocol.

SandRouter assumes that the provided data files were generated by dedicated scripts (see Section 5.3). In this sense, the format of IP addresses and the formats of updates should be checked. However, basic checks of both formats are provided in SandRouter. There are further checks for manipulation with IP address or accessing individual bits of an address.

The tests were performed with the nature of the update traffic. There can be cases where prefixes can be repeatedly announced to be withdrawn. Even though the prefix could have been already removed from both the RIB table and the FIB table. In this sense, the prefix cannot be found in either of the structures. The case of a search for a prefix that does not exist is tested and checked. However, this issue is not considered to cause the simulation to stop. These cases will pass without system failure and will not be included in the statistical outputs.

Dynamic Statistics and Structural Statistics can visualize different numbers of levels. This is caused by the implementation of Dynamic Statistics and the *UpdateAnalyser* class. This class gathers statistical data provided by methods that store or remove prefixes from structures of a FIB table. If there was no access by any update at some level, and there were no other levels that were accessed and modified beneath it, the level would not be visible in the Dynamic Statistics output.

5.4.1 Implementation Issues

During testing and experiments, a bug was observed for some input data. If two simulations are executed, one with the Binary Trie model and another with the Tree Bitmap model, and both simulations are provided with the same data, the Dynamic Statistics can show a different total value of Delete updates. This difference is not significant and does not affect the overall statistical output (in units).

5.5 Customization of the FIB Model

As part of the SandRouters solution and architecture, there is the possibility to include a user-defined FIB model implementation. The model can then be selected via the input parameter (see the list of input arguments at the beginning of Section 5.2) for the simulation. Each model, those introduced in Section 5.2.3, has to implement the same interface, so that other modules can interact with it. The following are declarations of these methods, also with the return values and further implementation descriptions.

- `def __init__(self):`

  Each model needs to implement an initialization method with no arguments accepted.

- `def SavePrefix(self, _addr: GeneralObjects.AddressIPv4|GeneralObjects.AddressIPv6) -> str:`

Each model needs to implement a method, by which is possible to store prefixes. There is no need to support any functionalities corresponding to IPv4 or IPv6 protocols. The object of *GeneralObjects.AddressIPv4* and *GeneralObjects.AddressIPv6* provide useful methods (see Section 5.2.5). This method needs to return a string value, that contains statistical data for Dynamic Statistics, which will comply with a predetermined format, described further in this section.

- `def DeletePrefix(self, _addr: GeneralObjects.AddressIPv4| GeneralObjects.AddressIPv6) -> str:`

  Same as for method *SavePrefix*, this one needs to remove provided prefix from a data structures. This method also needs to support as input arguments objects of the type *GeneralObjects.AddressIPv4* and *GeneralObjects.AddressIPv6*. The return value is the same as for the previous method.

- `def GetStatistics(self) -> list:`

  To let SandRouter collect statistic information about the structure of the FIB table, the model needs to implement this method. It is going to be called every x-th update (where x equals to an input argument **period**, see the list of input arguments at the beginning of Section 5.2), which will be propagated into the FIB module (see Figure 4.2). The output value is then stored in the Statistical Collector module and after the simulation, the data is processed and provided in Structural Statistics output. The format of the return value is described further in this section.

Return Format of SavePrefix and DeletePrefix Methods The format of a return value is as follows. The string contains the history of a walk through the FIB tables structure, checking if a new node had been created (or removed in the context of the DeletePrefix method) and at what level. The following is an example of a walk through, in which the format of one step is;

$$level\#action\_performed$$

The parameter *level* represents the level of the structure where the action was executed, starting from 0. The second parameter, $action\_performed$, indicates if the action was executed at the corresponding level (the structure was created or deleted, according to the context of the method). The parameter can hold a value 0 or 1, corresponding to a bool values True and False. Each step is separated from others by the sign |. The following line is an example, which can be interpreted in two ways. If the method that returned this string was the SavePrefix method, the message would be; The prefix was saved on the 5th level, and through the walk through, two nodes were created, at levels 4 and 5. Similar information could be interpreted if the method was DeletePrefix but with a different action.

$$|0\#0|1\#0|2\#0|3\#0|4\#1|5\#1|$$

Return Format of GetStatistics Method The type of the return value is a list of lists. These lists correspond to nodes. As the structure of a FIB table is scanned, the scanning must follow the Breadth-first search order. In other words, by levels from the top down. At each level, each node is checked for the following information. The first is how many prefixes it represents, and the second is how many children it has. This information is

stored in a list that is then added to the overall list. The format of a list representing one node is;

$$[number\_of\_prefixes, number\_of\_children]$$

As soon as all nodes from one level are processed, the end of the level is indicated by a list, which will contain only the value $None$. If no other levels exist to process, the last list will contain only a string 'END'. The following is the example of the return value from method *GetStatistics(...)*. This list shows that the structure has five levels. At the first and second levels, there is only one node. The second level node represents one prefix, and it has two children.

$$[[1,1],[None],[1,2],[None],[1,2],[1,1],[None],[1,0],[0,1],[1,1],[None],[1,0],[1,0],['END']]$$

5.6 Demonstration

This section demonstrates the use of SandRouter. The example includes a small FIB table, containing one prefix, and update traffic, containing four updates (see Figure 5.1). The demonstration is performed step by step to show the use of SandRouter framework with an explanation of the results. The IP protocol version utilized is IPv4. In demonstration, sample snapshot file and update file are used, which are included in folder *SandRouterApp/Tests/ipv4/*. In Listing 5.6 is shown the command that executes SandRouter framework in the root folder *SandRouterApp/*. Omitted arguments *-m* and *-p* inflict to use default values. The first corresponds to a FIB model for simulation, the Binary Trie algorithm. The second corresponds to the default interval between analyses of the entire FIB data structure, which is set to zero. In the context of a SandRouter, this means that analysis of the FIB data structure will be performed after every update.

```
snap="./Tests/ipv4/snapshot_example.txt"
upds="./Tests/ipv4/updates_example.txt"
dst="../example_out/"
./SandRouter.py -s $snap -u $upds -d $dst -v ipv4
```

Listing 5.6: The example of the execution of SandRouter.

The content of the *snapshot_example.txt* and *updates_example.txt* files is as follows:

```
snapshot_example.txt:
0.0.0.0/0
---------------
updates_example.txt:
A 0.0.0.0/1
A 255.0.0.0/1
A 255.0.0.0/1
W 0.0.0.0/1
```

Figure 5.1: The content of the input files of the demonstration example.

As described in Figure 4.2, after all input arguments have been processed, the RIB table and the FIB table are initialized. The snapshot file contains only one prefix (see Figure 5.1). This means that at the start of the simulation, the FIB table contains one prefix node, which also represents a root of the whole data structure. After initialization comes the processing of update. As a reminder of the convention, the A symbol at the beginning

of the update means that the following prefix is going to be stored in the RIB table, or alternatively in the FIB table (see Algorithm 1). The symbol W represents the deletion of the prefix. The first two updates cause the creation of two nodes because the FIB table contains only one node (root). These two nodes will be direct children of the root node. At this moment, the FIB table can be vizualized as a tree consisting of three nodes - one parent and two children.

The next update is duplicated because the prefix information is already stored in both tables. At this moment, the update will propagate only in the RIB table. In the context of SandRouter, these updates are understood as alternative paths to the network, corresponding to this prefix (see Section 2.3). The last update will cause deletion of not only the prefix but also the whole node. After all updates are processed, the simulation terminates and the final results will be generated in the folder provided at the execution of SandRouter.

Now, the results can be described, which have been generated after the simulation. The Dynamic Statistics shows all updates propagated in the FIB table. The output is divided into three columns. The first column corresponds to the statistics of saving updates. The statistic is shown as the list of levels that were accessed by updates. If the update causes a creation of a node at some level, the information is visualized here. The updates that changed the level are added up for each level. The total numbers are then shown at the end of the simulation. Below the list are provided the total number of all saving updates and the total number of nodes created during the simulation. The second column is the same list of statistics. However, data are collected for deletion of updates. The third column shows the minimum and maximum total number of nodes at each level throughout the simulation.

```
-------------------------------------------------------------------------------
     |    SAVE UPDATES    |    DELETE UPDATES    |   The total number of nodes
-------------------------------------------------------------------------------
Level|   Updates      %  |      Updates      %  |        Min       Max
  0. |      0         0% |         0         0% |         1         1
  1. |      2       100% |         1       100% |         0         2
-------------------------------------------------------------------------------
     Number of updates:  2|Number of updates:  1 |
     Created nodes:  2    |Removed nodes:  1
```

Figure 5.2: Dynamic Statistics of the demonstration example.

Based on the results shown in Figure 5.2, the first two updates can be seen. There are two updates that accessed level 1. With these two updates, two nodes were created. The duplicated update is not included. The last update removed one of the prefixes. The deletion has a further effect to delete the whole node, as can be seen in the second column. The last column shows the dynamicity of changes, by visualizing the minimal and maximal values of nodes, which were seen throughout the simulation.

The Structural Statistics shows the information gathered by the *GetStatistics(...)* method (see Section 5.2.4). The structure of the FIB table is scanned by levels, inspecting multiple properties (see Section 4.3). The results are then presented as graphs. Each graph consists of an x-axis, correlated with the level of the FIB data structure, or a mask of stored prefixes. The y-axis corresponds to the number of objects seen at each level. The graphs in Figures 5.3 and 5.4 show the maximum value of prefixes and children, found at each level throughout the simulation. In the case of the Binary Trie algorithm, the graphs do not

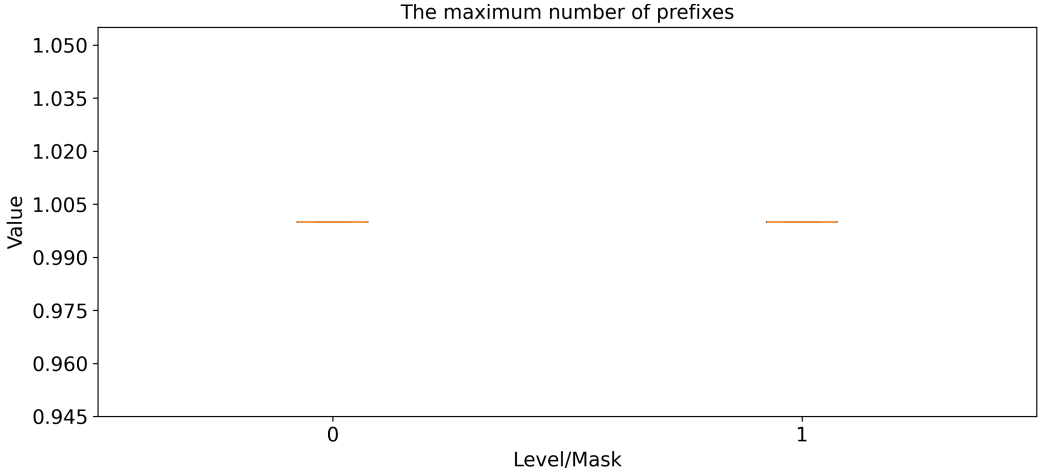

Figure 5.3: The graph of the macimum number of prefixes. The used FIB model was the Binary Trie algorithm.

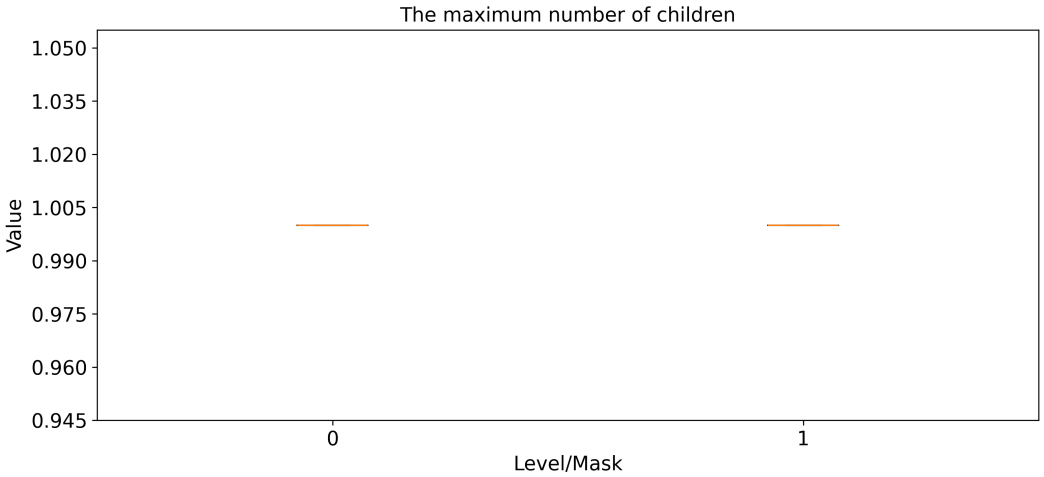

Figure 5.4: The graph of the maximum number of children. The used FIB model was the Binary Trie algorithm.

show much of the interesting results. The situation could be different for different LPM algorithms, such as the Tree Bitmap algorithm (see Section 3.3.3).

Before the process of updates, an analysis of the data structure of the FIB table is performed. In this sense, the changes are collected in view of the state at the beginning of the simulation. The example can be seen in Figure 5.5, where the change in the number of leaves is displayed at each level. Based on the updates shown in Figure 5.1, it can be interpreted that there is a decrease in the number of leaves at level 0 and growth at level 1. For the other two graphs in Figures 5.6 and 5.7, no drop occurred at level 0. The number of nodes and prefixes at level 0 remained at value 1. However, the growth is visible at level 1.

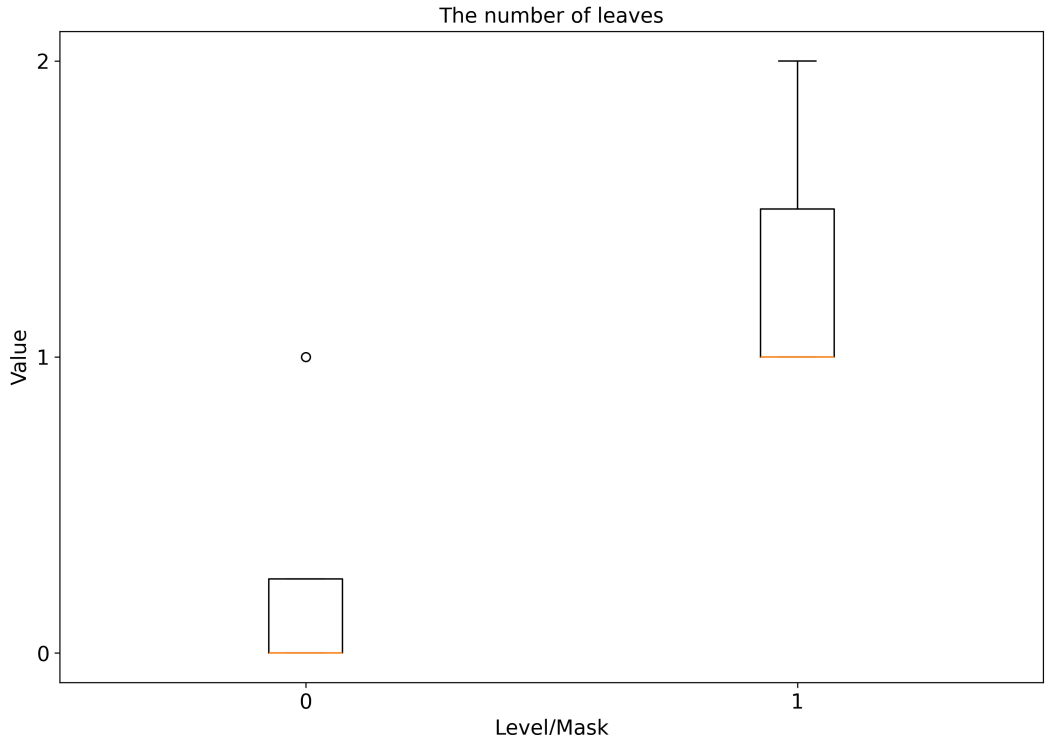

Figure 5.5: The graph of the number of leaves. The used FIB model was the Binary Trie algorithm.

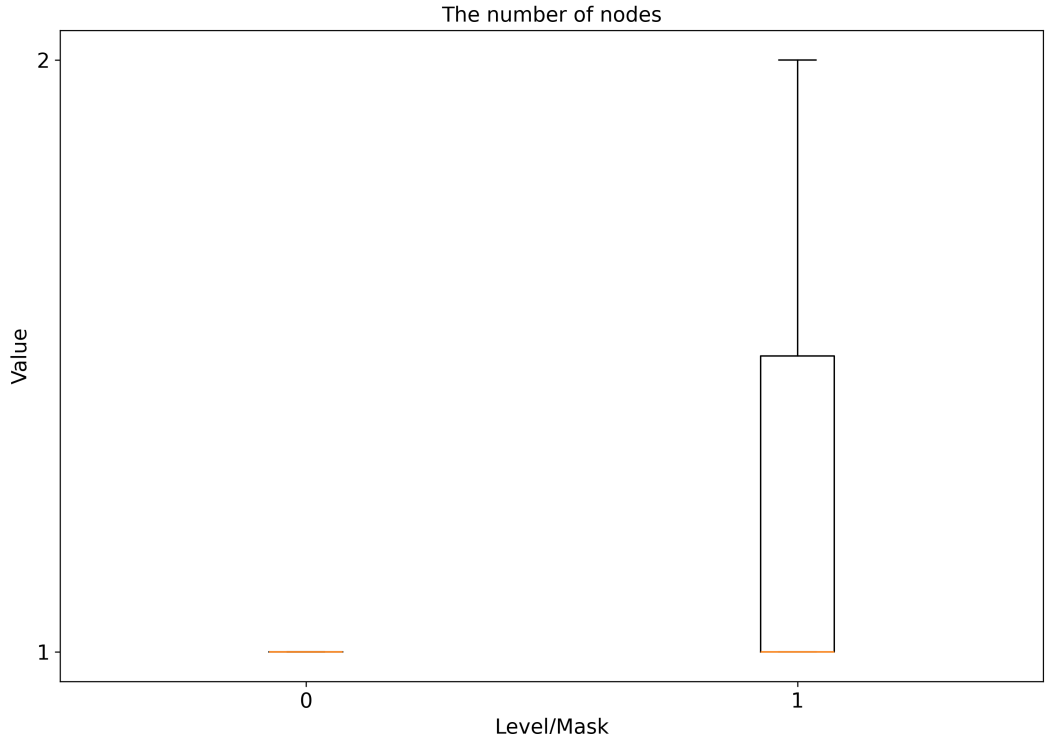

Figure 5.6: The graph of the number of nodes. The used FIB model was the Binary Trie algorithm.

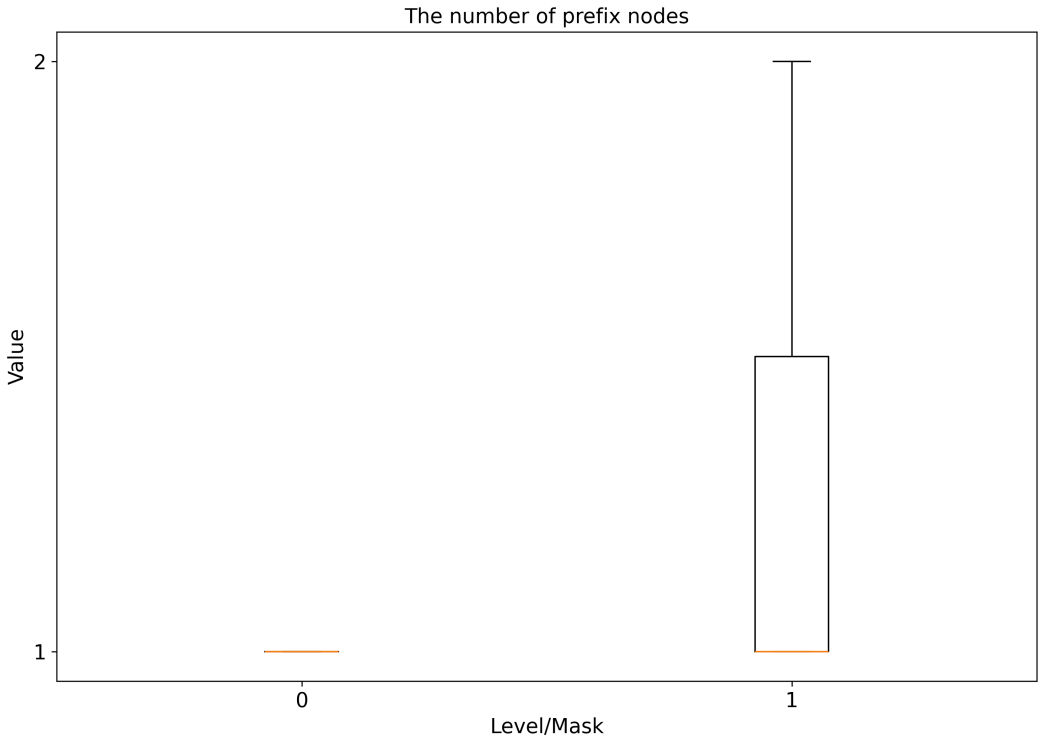

Figure 5.7: The graph of the number of prefix nodes. The used FIB model was the Binary Trie algorithm.

Chapter 6

Experiments and Results

At this point, SandRouter can demonstrate its ability to examine the dynamic of changes within the FIB tables. All examined tables are inside BGP collectors, located in the core of the Internet network. Access to these collectors is provided by communities founded by universities or groups around the RIPE NCC (Réseaux IP Européens Network Coordination Centre). In these experiments, we used data provided by the RouteViews[1] group and Routing Information Service[2] (RIS) of the RIPE NCC community. This is also the Regional Internet Registry for Europe, the Middle East, and Central Asia[28].

All experiments were performed with two FIB table algorithms, implemented in the SandRouter framework: Binary Trie and Tree Bitmap with $SL = 3$. SandRouter was implemented to support both IPv4 and IPv6 prefixes. However, it can process only one version of the IP protocol in one simulation. In this way, the input files were pre-processed using dedicated scripts. The following experiments describe how the SandRouter could be used.

6.1 Experiment 1

The first experiment was carried out to see what can be found in the resulting statistics from SandRouter, considering only IP protocol IPv4. This experiment was then enhanced to show what the results look like for different FIB tables, located in routers of different data sources, different locations, and different dates when updates were taken. Four routers were randomly selected, two from each of the data sources (see Table 6.1). Because the results turned out to be very similar, only one test is described in detail. The output of each simulation consists of four box plot graphs, displaying Structural Statistics, and one text file, with Dynamic Statistics. For each collector, the simulation was run twice. First, with a FIB table implemented as Binary Trie, and then with the implementation of the Tree Bitmap. For simulations, 70 minutes of the traffic data set was used, from 1. 4. 2024. The following outputs correspond to the collector managed by RIS, located in London.

[1] https://www.routeviews.org/routeviews/index.php/about/

[2] https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris/

[5] https://archive.routeviews.org/route-views.linx/

[6] https://archive.routeviews.org/route-views.saopaulo/

Data Srource	Evaluated Algorithm	Record Date	Time Period of Simulated Updates
RouteViews, LINX[3]	Binary Trie, Tree Bitmap	2023.05.13	70 minutes
RouteViews, SAOPAULO[4]	Binary Trie, Tree Bitmap	2021.02.27	70 minutes
RRC01[5]	Binary Trie, Tree Bitmap	2024.04.01	70 minutes
RRC14[6]	Binary Trie, Tree Bitmap	2023.12.07	70 minutes

Table 6.1: Overviw of information of Experiment 1.

6.1.1 FIB Table as Binary Trie

From the simulation with Binary Trie, the structure appeared static. Not many dynamic changes were visible on any of the graphs (all boxes seem to be squeezed around a few values). However, based on Dynamic Statistics (see Figure 6.1), we can see that all updates accessed levels 17 to 32. This may be due to how the CIDR concept evolved and works today (see Section 2.3.2 and Figure 2.6).

	SAVE UPDATES		DELETE UPDATES		The total number of~nodes	
Level	Updates	%	Updates	%	Min	Max
0.	0	0%	0	0%	1	1
1.	0	0%	0	0%	1	1
2.	0	0%	0	0%	2	2
3.	0	0%	0	0%	4	4
4.	0	0%	0	0%	7	7
5.	0	0%	0	0%	14	14
6.	0	0%	0	0%	28	28
7.	0	0%	0	0%	56	56
8.	0	0%	0	0%	112	112
9.	0	0%	0	0%	215	215
10.	0	0%	0	0%	423	423
11.	0	0%	0	0%	822	822
12.	0	0%	0	0%	1575	1575
13.	0	0%	0	0%	2956	2956
14.	0	0%	0	0%	5542	5542
15.	0	0%	0	0%	10200	10200
16.	0	0%	0	0%	18354	18354
17.	4	1%	4	1%	26065	26067
18.	13	2%	12	2%	42773	42778
19.	17	2%	16	3%	67829	67836
20.	47	7%	43	7%	104206	104217
21.	71	10%	61	10%	148246	148264
22.	93	14%	68	11%	212970	212999
23.	199	29%	154	24%	270342	270425
24.	350	51%	268	42%	366200	366298
25.	72	11%	73	12%	2880	2890
26.	47	7%	48	8%	3025	3032
27.	41	6%	42	7%	3383	3388
28.	42	6%	44	7%	3936	3941
29.	55	8%	60	10%	4386	4393
30.	60	9%	71	11%	4450	4461
31.	90	13%	103	16%	5865	5879
32.	142	21%	158	25%	8637	8655

Number of~updates:684|Number of~updates:631 |
Created nodes:1343 |Removed nodes:1225
min depth: 17 |min depth: 17
max depth: 32 |max depth: 32

Figure 6.1: Dynamic Statistics of a FIB table implemented as Binary Trie.

The busiest with respect to both saving and deleting updates is level 24. As can be seen, half of the saving updates (51%) changed this level. In contrast, nearly half of the deletion updates (42%) also changed this level. Even with this, the difference between this level's maximum value and the minimum value of nodes is 98. Compared with the maximum value, the difference value is an order of magnitude lower. This means that in the resulting graphs, the fluctuation of the values is not as obvious (see Figure 6.2).

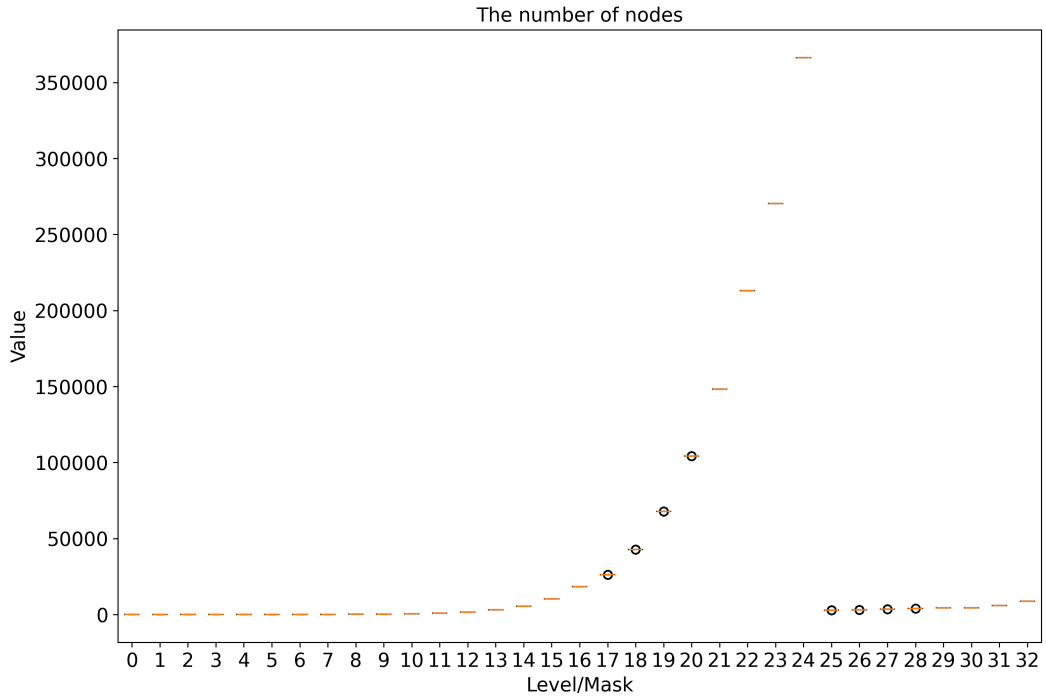

Figure 6.2: The graph with the number of nodes, results of simulation with the use of the Binary Trie algorithm.

The graph in Figure 6.3 also shows another fact. Based on the graph, levels 1 to 7 do not store any prefixes. This was the same for all the results of all simulations. This is due to the distribution of IPv4 address space between Regional Internet Registries (RIR), as was mentioned in Section 2.3.4. The IP address space was divided into 256 blocks (based on the first octet of the IPv4 address). Each block is now managed by some RIR. The list of assignments is managed by IANA [18].

From a structural point of view, the graphs correspond to what we expect from the Binary Trie algorithm. As mentioned in Section 3.2, the structure is a simple binary tree with a depth equal to 32. The exponential growth of nodes with each level (except for the last seven levels) corresponds to the expected behavior (see Figure 6.2). The reason why lower levels are not as filled as level 24 (below level 24 in the trie) most likely corresponds to how IPv4 addresses were assigned over time. With the evolution from classful addressing scheme to CIDR convention (see Sections 2.3.2), this may also be the reason why all updates only targeted levels 17 to 32 (see Figure 6.1). The comparison of results in Figure 6.4 and Figure 6.2 shows that the large group of nodes at level 24 is mostly leaf nodes. This corresponds to the results in Figure 6.5. It is obvious that all leaf nodes in Figure 6.4 are also prefix nodes in Figure 6.5. However, the opposite cannot be determined. If we look at levels higher than level 24 in Figure 6.5, the number of prefixes is greater than the number of leaf

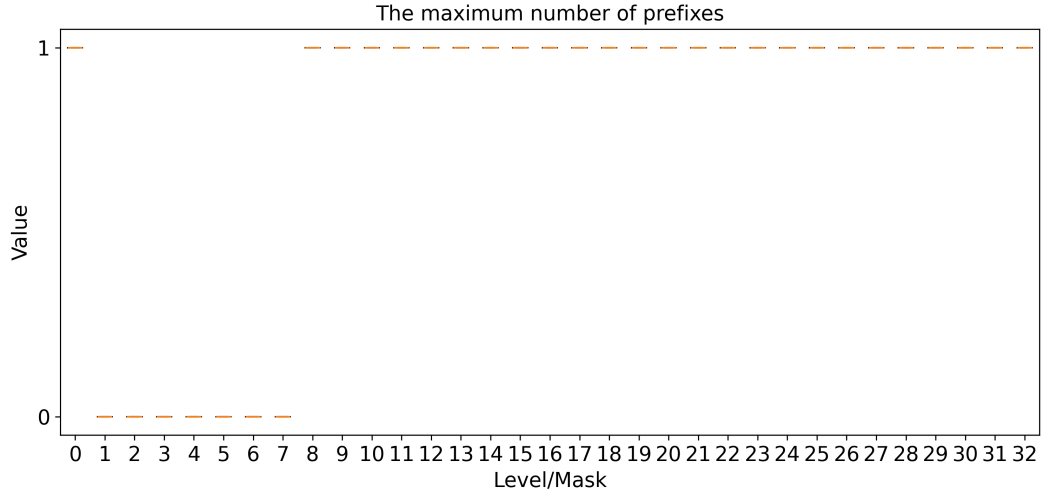

Figure 6.3: The graph with the maximum number of prefixes, results of simulation with the use of the Binary Trie algorithm.

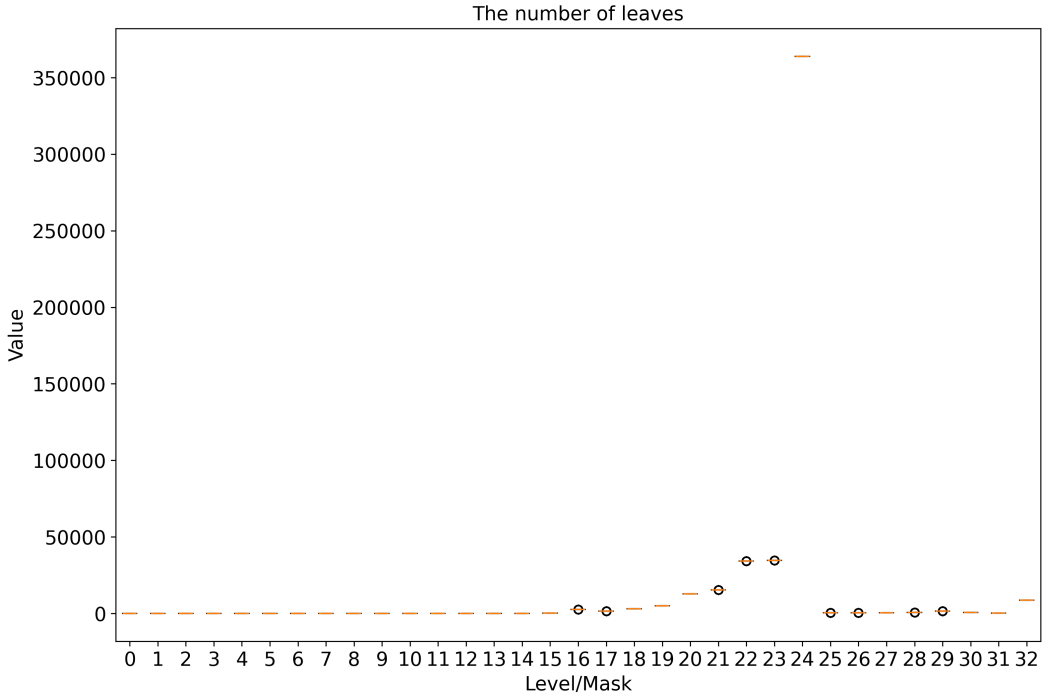

Figure 6.4: The graph with number of leaf nodes, results of simulation with the use of the Binary Trie algorithm.

nodes. Still, most of the nodes on these levels are empty (for example, levels 21, 22, and 23).

The last two graphs are an interesting way to visualize the general structure of a node at each level. For example, consider theoretical scenarios where the worst case of memory use needs to be known. The combination of Figure 6.6 and Figure 6.3 can show important facts. All nodes, except for the node at level 32, need to save information about their children. In addition, most of them also need to have space to store the prefix information.

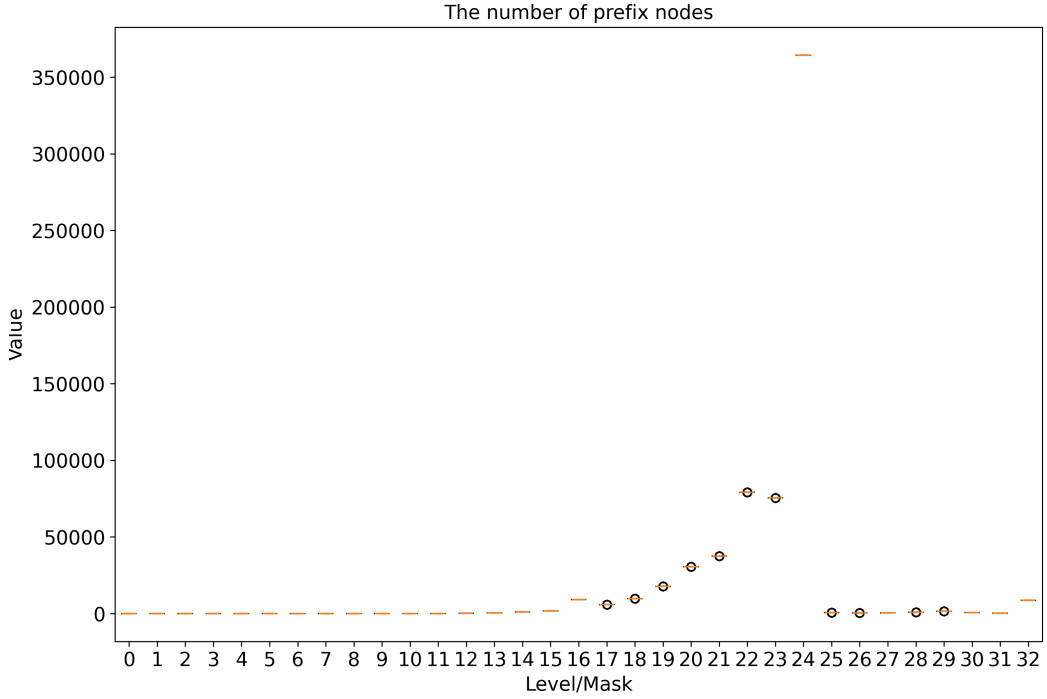

Figure 6.5: The graph with number of prefix nodes, results of simulation with the use of the Binary Trie algorithm.

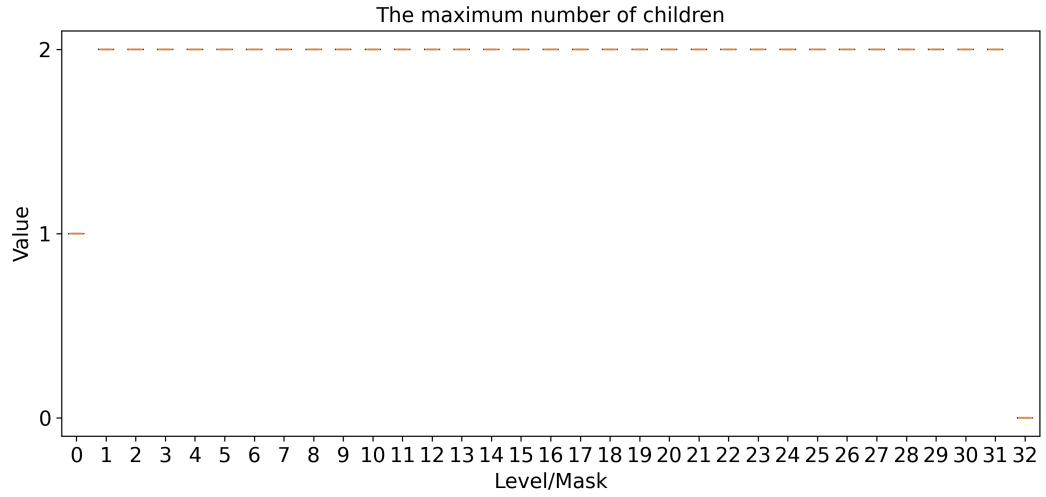

Figure 6.6: The graph with the maximum number of children, results of simulation with the use of the Binary Trie algorithm.

If we look at Figure 6.3, another factor is also visible that corresponds to the management of the IPv4 address space. For levels 1 to 7, and with the addition of level 0, which contains only one prefix, $*$, this corresponds to a first octet of IPv4 addresses. This is also a portion that corresponds to prefixes of a Class A number space (see 2.3.2). Based on IP address management, the first octet of an IPv4 address corresponds to an identifier of the IP block, which is then managed by one of the Regional Internet Registries (RIRs). Consequently,

the first 8 levels (ranging from number 0 to number 7) do not contain prefixes, and there have been no recorded updates accessing them.

6.1.2 FIB table as Tree Bitmap

The second simulation was performed with the use of the Tree Bitmap algorithm. The same data set was used and the same format of results was generated. With the use of TBM, the structure is more compact. The number of levels is much smaller, compared with a Binary Trie. This is because TBM uses a more complicated node structure, which can compress a group of nodes into a single one. The number of children for one node is up to 8 and the number of stored prefixes is up to 7 (see Section 3.3.3). In this way, the graphs in Figure 6.8 and in Figure 6.9, are different from the graphs seen for the Binary Trie algorithm (see Section 6.1). Finally, there is also visible a dynamic change. At level 11 of the graph in Figure 6.9, the maximum value of the prefixes changed from 1 to 4, with the median at 3. Next, for the graph in Figure 6.8, there was a slight change at level 10. For the rest of the graphs, the results are similar to the previous test run.

	SAVE UPDATES		DELETE UPDATES		The total number of~nodes	
Level	Updates	%	Updates	%	Min	Max
0.	0	0%	0	0%	1	1
1.	0	0%	0	0%	4	4
2.	0	0%	0	0%	28	28
3.	0	0%	0	0%	221	221
4.	0	0%	0	0%	1604	1604
5.	0	0%	0	0%	10556	10556
6.	12	2%	11	2%	45999	46003
7.	66	10%	56	9%	170375	170394
8.	308	45%	231	39%	425625	425742
9.	31	5%	31	5%	3872	3878
10.	60	9%	70	12%	5923	5933
11.	126	18%	140	23%	8640	8655

Number of~updates:684	Number of~updates:596
Created nodes:603	Removed nodes:539
min depth: 6	min depth: 6
max depth: 11	max depth: 11

Figure 6.7: Dynamic Statistics of a FIB table implemented as Tree Bitmap model.

6.1.3 Summary

Based on the results shown in Experiment 1, we formulate the following conclusion; All tests show that the Structural Statistics of the FIB tables are similar, regardless of the place of the collector or time. There were two dynamic changes when the TBM implementation of a FIB table was used. Based on the results of Dynamic Statistics, there were small differences between the time intervals and collectors. This could have happened because of the nature of the passing BGP communication, at that moment, in that location. However, these changes did not affect the overall structures shown in the graphs. We believe that the structures studied were still too robust to show any dynamic changes. Even though the changes occurred (based on the Dynamic Statistics). The previous results proved that SandRouter is capable of showing dynamic changes in FIB table structures. The issue of displaying and recognizing them in output graphs can be caused by changes being too small and therefore not significant enough.

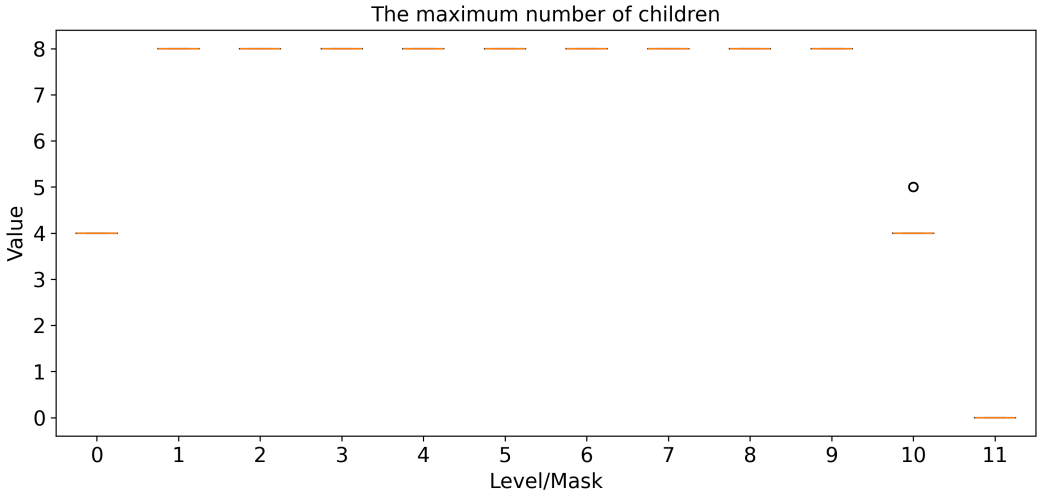

Figure 6.8: The graph with the maximum number of children, results of simulation with the use of the Tree Bitmap algorithm.

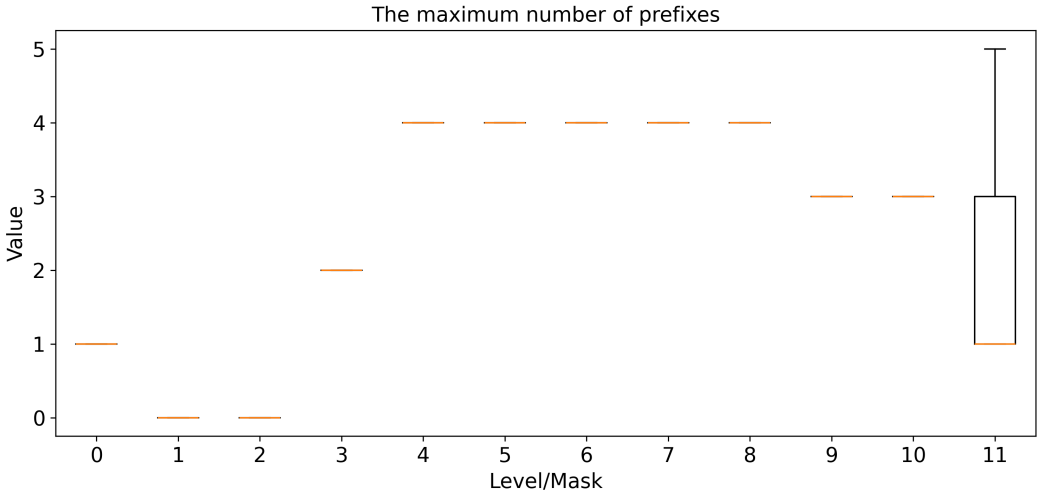

Figure 6.9: The graph with the maximum number of prefixes, results of simulation with the use of the Tree Bitmap algorithm.

6.2 Experiment 2

The second experiment was carried out based on the conclusion introduced in Section 6.1. Before each simulation, the content of the snapshot file and the update file was filtered according to the selected Regional Internet Registry (see Section 2.3.4). As described in Section 2.3.2, the IP address space is divided and managed by multiple regional registries. In this sense, we try to divide the input traffic of prefixes according to the registry that manages them. These registries are ARIN, AFRINIC, RIPE NCC, APNIC, and LACNIC. The whole address space is divided into 256 groups. Each group can be distinguished using the first octet of the IPv4 address. The list of assignments is provided on the dedicated website [18]. The traffic snapshots utilized as sample data were used from the following BGP collectors:

- RRC01[7] - London
- RRC11[8] - New York
- RRC19[9] - Johannesburg
- RRC23[10] - Singapore
- RRC24[11] - Montevideo

70 minutes of updates were downloaded from each collector, from the same date and time, 1. 4. 2024, starting from midnight. The group of tested collectors includes those that are located in different regions. The same as in Section 6.1, each simulation was performed twice, one with the implementation of Binary Trie and one with the implementation of Tree Bitmap.

By dividing the structure of the FIB table into smaller parts (based on the selected registry), the number of updates that affect the respective *subtree* of a FIB table is much smaller. The subtree in this experiment corresponds to the part of a FIB table, which shows only the prefixes managed by one registry. In this sense, we assume that the information in the resulting statistics will not be too overwhelming, thereby they will show the dynamicity of changes more clearly.

In other words, our aim is to demonstrate that by focusing on a smaller part of the overall FIB table structure, SandRouter is better able to show the dynamic nature of updates. In the following part, the results of the second experiment are introduced. Based on the description structure of Section 6.1, the results of simulations employing a Binary Trie model for the FIB table are presented first. After that, results from simulations employing the Tree Bitmap algorithm are then presented. However, because of the number of resulting files from all simulations, only the most interesting cases are described. If there is an interest to see all the results of this experiment, follow the tutorial provided in README file, located in *SandRouter/* folder (see Appendix A with content of Storage Medium).

[7] https://data.ris.ripe.net/rrc01/
[8] https://data.ris.ripe.net/rrc11/
[9] https://data.ris.ripe.net/rrc19/
[10] https://data.ris.ripe.net/rrc23/
[11] https://data.ris.ripe.net/rrc24/

6.2.1 FIB Table as Binary Trie

According to Dynamic Statistics, the traffic of updates corresponding to the AFRINIC registry was the least dense among all collectors. In contrast, most updates appeared in subtrees, corresponding to the RIPE NCC registry and ARIN registry. From the inspection of each group of results, one phenomenon was similar in each case. If the location of a collector was the same as the location of the inspected registry, for example, collector rrc24 is located in Uruguay, Latin America, and the inspected registry was LACNIC, the results of Dynamic Statistics show the highest dynamic from all collectors from other regions. In addition, the part of the FIB table corresponding to this region, at this collector, was the most robust and deep. This was not visible in the results of Experiment 1 (see Figure 6.2), because the structure of FIB table was inspected as a whole in all regions.

	SAVE UPDATES		DELETE UPDATES		The total number of~nodes	
Level	Updates	%	Updates	%	Min	Max
0.	0	0%	0	0%	1	1
...	...	...	...	...	...	...
8.	0	0%	0	0%	5	5
9.	0	0%	0	0%	6	6
10.	0	0%	0	0%	12	12
11.	0	0%	0	0%	24	24
12.	0	0%	0	0%	48	48
13.	0	0%	0	0%	93	93
14.	0	0%	0	0%	176	176
15.	0	0%	0	0%	330	330
16.	0	0%	0	0%	635	635
17.	0	0%	0	0%	1040	1040
18.	0	0%	0	0%	1622	1622
19.	1	0%	1	0%	2345	2346
20.	9	3%	9	3%	3476	3485
21.	34	10%	34	10%	5273	5306
22.	70	20%	70	20%	7651	7718
23.	162	47%	162	47%	8993	9142
24.	252	73%	251	73%	13324	13547
25.	0	0%	0	0%	48	48
26.	0	0%	0	0%	73	73
27.	0	0%	0	0%	114	114
28.	0	0%	0	0%	166	166
29.	1	0%	1	0%	207	208
30.	1	0%	1	0%	182	183
31.	1	0%	1	0%	197	198
32.	1	0%	1	0%	325	326

Number of~updates:344|Number of~updates:345 |
Created nodes:532 |Removed nodes:531
min depth: 21 |min depth: 21
max depth: 32 |max depth: 32

Figure 6.10: The output of Dynamic Statistics from simulation using Binary Trie. The input files originated from collector rrc19 (see Section 6.2). Data specifically focused on AFRINIC registry.

As examples, the following results show the outputs corresponding to the AFRINIC registry, compared to the outputs corresponding to the RIPE NCC registry. The output shows that the most traffic, corresponding to the AFRINIC registry, passed through the collector rrc19 located in South Africa (see Figure 6.10). This is also the only collector in which the updates reached levels lower than level 24. In the output from the collector rrc01 (and for all other regions), no update reached a lower level than level 24 (see Figure

6.11). As seen in Experiment 1, a higher dynamic is visible at level 24. From this, it can be considered that this is due to passing traffic, at that moment, at that location. However, if we look at the graphs of structural statistics from other regions, in fact, the size and number of levels of subtrees also differ. This did not appear in the results of Experiment 1, because the differences were hidden in the entire structure of the FIB table. As an example, see Figure 6.12 and Figure 6.13. Both are visualizations of the number of prefix nodes, but each from a different collector located in a different region. The first figure contains a graph of a subtree originating from a collector located in Johannesburg. The second figure shows the results of a collector located in London, which is also the collector with most of the bussies traffics from all tested collectors. The number of levels for the same set of prefixes (in the context of RIR) differs based on the location where the subtree is inspected. The same phenomenon was seen for the LACNIC registry. The subtree contained the most nodes, at the Montevideo location. The following Table 6.2 contains the extracted values measured by SandRouter. The table contains the minimal and the maximal values of nodes of levels 22, 23, and 24, also with the number of levels visualized in structural statistics. These values were filled in for each of the collectors tested.

	SAVE UPDATES		DELETE UPDATES		The total number of~nodes	
Level	Updates	%	Updates	%	Min	Max
0.	0	0%	0	0%	1	1
...	...	...	...	...	...	...
9.	0	0%	0	0%	6	6
10.	0	0%	0	0%	12	12
11.	0	0%	0	0%	24	24
12.	0	0%	0	0%	48	48
13.	0	0%	0	0%	93	93
14.	0	0%	0	0%	176	176
15.	0	0%	0	0%	330	330
16.	0	0%	0	0%	634	634
17.	0	0%	0	0%	1046	1046
18.	0	0%	0	0%	1661	1661
19.	0	0%	0	0%	2364	2364
20.	2	9%	2	9%	3531	3533
21.	7	32%	7	30%	5348	5354
22.	1	5%	1	4%	7817	7818
23.	4	18%	4	17%	9257	9258
24.	13	59%	12	52%	13739	13743

Number of~updates: 22	Number of~updates: 23
Created nodes: 27	Removed nodes: 26
min depth: 21	min depth: 21
max depth: 24	max depth: 24

Figure 6.11: The output of Dynamic Statistics from simulation using Binary Trie. The input files originated from collector rrc01 (see Section 6.2). Data specifically focused on AFRINIC registry.

6.2.2 FIB Table as Tree Bitmap

As was already discovered in Experiment 1 (see Section 6.1), the structures used by the Tree Bitmap algorithm are better suited to show the dynamicity of changes in the structure. The best case was the graph with the maximum number of prefixes (see Figure 6.9). As mentioned in the previous section, the most dynamic structure is the subtree corresponding to the RIPE NCC registry. Based on this, the results of this case are further inspected.

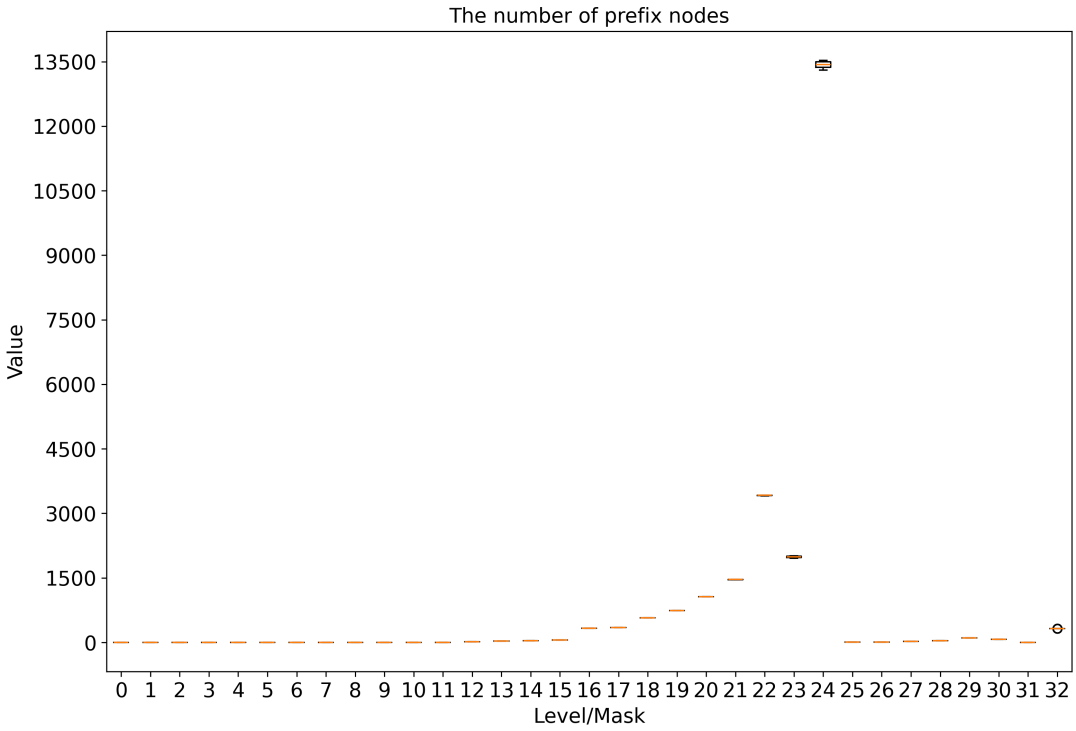

Figure 6.12: The graph showing the number of prefix nodes in simulation using Binary Trie algorithm, sourced from collector rrc19 (see Section 6.2). Data specifically focused on AFRINIC registry.

Collector/Region	**Min Nodes Number**			**Max Nodes Number**			**The Levels Number**
	Lvl 22	**Lvl 23**	**Lvl 24**	**Lvl 22**	**Lvl 23**	**Lvl 24**	
RRC01/London	17456	24160	32428	17457	24165	32446	29
RRC11/New York	16659	22201	29459	16666	22217	29493	32
RRC19/Johannesburg	17360	23819	31545	17361	23824	31559	24
RRC23/Singapore	17427	24049	32284	17429	24055	32299	32
RRC24/Montevideo	19283	25752	35489	19289	25763	35504	32

Table 6.2: The table of selected result values, corresponding to the LACNIC registry prefix space. The **Lvl** stands for Level. The Min and Max node numbers were extracted from the Dynamic Statistics outputs and the numbers of levels of the subtrees from Structural Statistics.

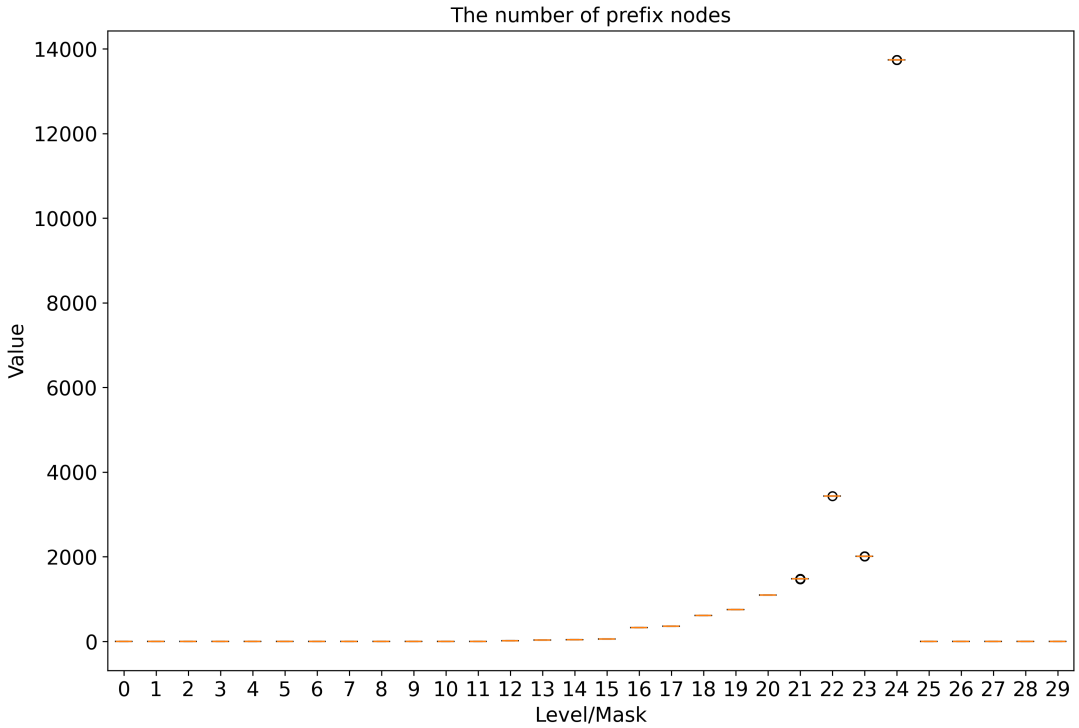

Figure 6.13: The graph showing the number of prefix nodes in simulation using the Binary Trie algorithm, sourced from collector rrc01 (see Section 6.2). Data specifically focused on AFRINIC registry.

From Figures 6.14, 6.15, and 6.16, which correspond to collectors in London, New York, and Singapure, the changes are noticable in all these cases.

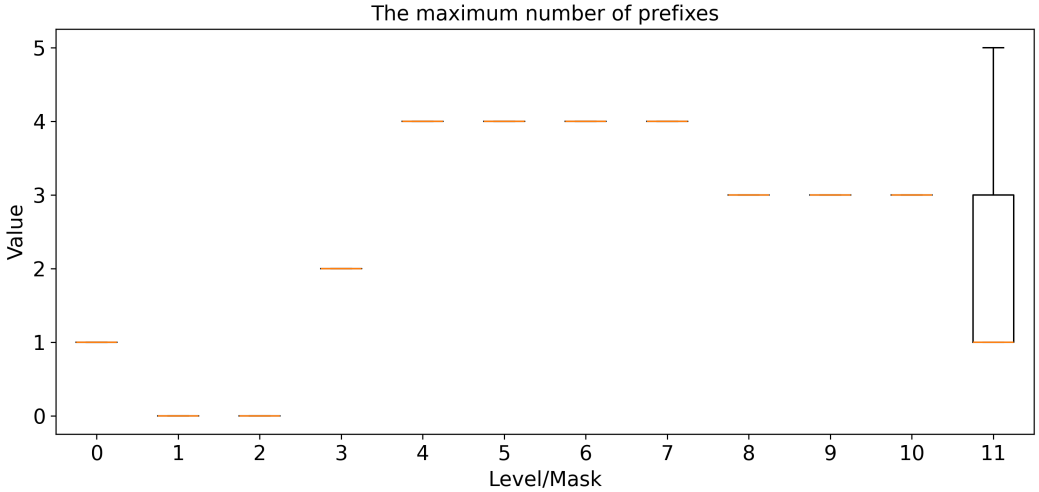

Figure 6.14: The graph of the maximum number of prefixes, originating from London.

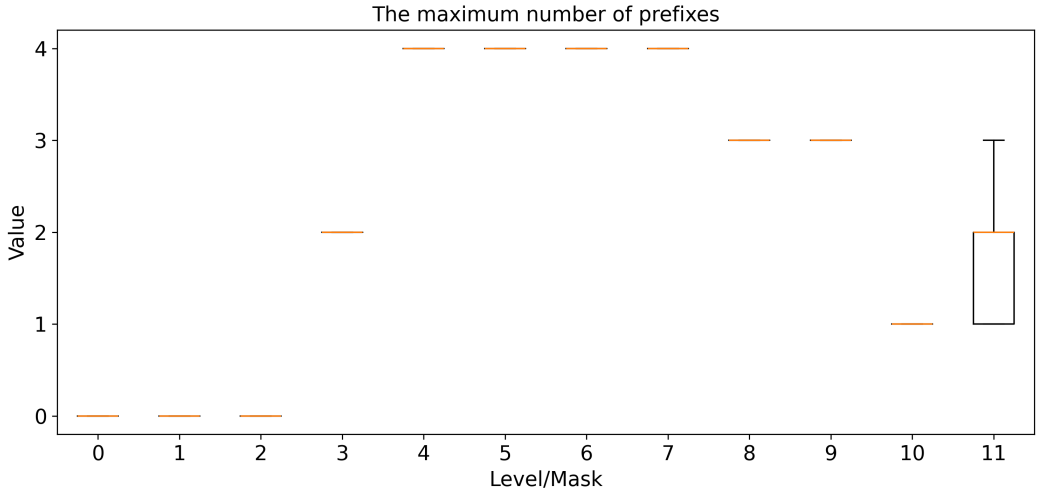

Figure 6.15: The graph of the maximum number of prefixes, originating from New York.

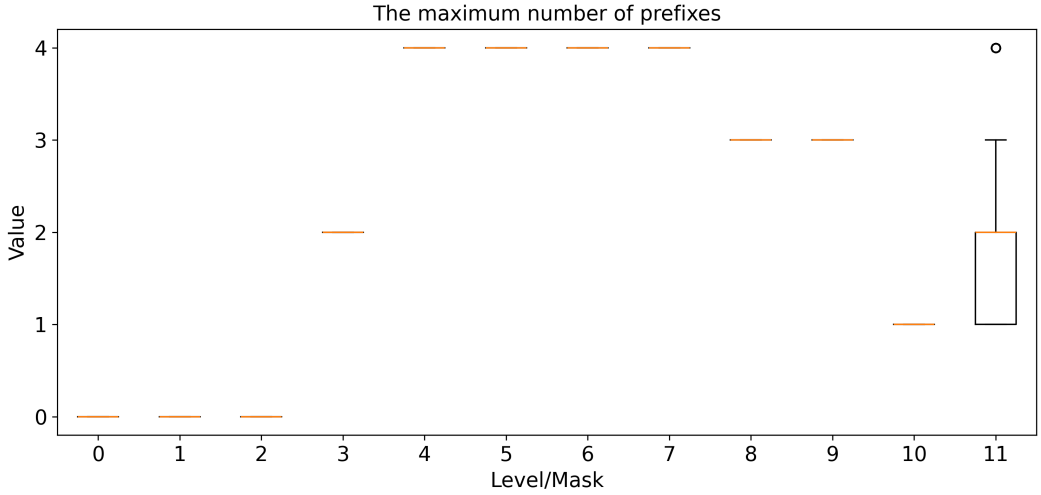

Figure 6.16: The graph of the maximum number of prefixes, originating from Singapore.

6.2.3 Summary

In this part, we summarize the results of Experiment 2. Based on an analysis of structural statistics, it was discovered that the height of the subtree corresponding to a certain registry may vary for each collector in different regions. In the context of Dynamic Statistics, the traffic and dynamicity of changes connected with a certain registry in certain regions can be much higher than for other registries.

The described results show that SandRouter is more capable of visualizing and detecting changes in the structures of the FIB table if it focuses on certain groups of prefixes. As mentioned in Section 6.1, the issue of displaying and recognizing changes in output graphs can be caused by changes being too small and therefore not significant enough. Based on the graphs presented in Section 6.2.2, the focus on a smaller group of prefixes caused the graph to show more evident dynamic changes in structural statistics. In this experiment, we decided to divide the traffic according to the RIRs. However, a different criterion can be used.

6.3 Experiment 3

In the previous experiments, the developed and studied FIB table data structure contained only IPv4 prefixes. This experiment focuses only on the IPv6 protocol. The structure of the experiment is similar to that of Experiment 1 (see Section 6.1). Four collectors will be selected, two from each data source (see the opening paragraphs of this chapter). The time for each collector is randomly selected. Because the size of a FIB table data structure will be much larger than in the previous experiments, the interval of updates will not exceed 60 minutes. With a higher number of updates (more than one million), the simulation can take more than 5 hours (see Section 7.1). The following table 6.3 shows the list of collectors tested for Experiment 3.

Data Srource	IP Protocol	Evaluated Algorithms	Record Date	Time Period of Simulated Updates
RouteViews, SOXRS[12]	IPv6	Binary Trie, Tree Bitmap	2024.03.16	60 minutes
RouteViews, TELXATL[13]	IPv6	Binary Trie, Tree Bitmap	2023.02.02	60 minutes
RRC01[14]	IPv6	Binary Trie, Tree Bitmap	2024.04.01	20 minutes
RRC23[15]	IPv6	Binary Trie, Tree Bitmap	2024.01.01	25 minutes

Table 6.3: Overviw of information of Experiment 3.

Because the IPv6 protocol has no evolution history, such as the IPv4 protocol, there was no prior idea of what the results could look like. Now, the situation is quite similar to that seen in previous experiments. Except for the number of levels, which for this experiment can be up to 128 levels. As seen in IPv4, the first 23 bits of the IPv6 address are used for the identification address block, managed by one of the RIRs (see Section 2.3.4).

6.3.1 FIB Table as Binary Trie

In the Dynamic Statistics, a high dynamicity is visible at levels 47 and 48. As can be seen in Figure 6.17, the number of levels that are accessed by updates is much higher than what

[12] https://archive.routeviews.org/route-views.soxrs/
[13] https://archive.routeviews.org/route-views.telxatl/
[14] https://data.ris.ripe.net/rrc01/
[15] https://data.ris.ripe.net/rrc14/

we have seen in previous experiments. However, similarly to IPv4, for levels lower than 48 (levels below level 48 in the trie), the dynamicity is much lower or even none. The structural statistics do not show much dynamic behavior. However, the similarity in the distribution of nodes and prefixes in the FIB table structure is quite similar to IPv4. As you can see in Figure 6.18 and Figure 6.19, the growth is visible until level 48. Then the values are nearly zero. Because the simulation output contains similar results, only those from collector RRC01 are provided. This collector is also the one with the most traffic.

6.3.2 FIB Table as Tree Bitmap

Based on the results, the SandRouter framework was able to show the dynamicity of the graph, showing the maximum number of prefixes. The same case was noticed in Experiment 2 (see Section 6.2). In Figure 6.20 two levels, 11 and 15, are visible to contain the change.

6.3.3 Summary

From the results seen in this section, the structure of the FIB table containing IPv6 prefixes indicates properties similar to those noticed in Experiment 1 (see Section 6.1). There was visible a significant growth of dynamicity around levels 47 and 48. More dynamicity appeared if for simulation the Tree Bitmap algorithm was used.

6.4 Summary of Experiments

All experiments show results proving that the SandRouter framework can visualize changes in the FIB table data structure. These changes can be visualized in two ways; by Dynamic Statistics, or by Structural Statistics. However, the first problem that came up with the first experiment was that the changes on each level were too small to be noticed in the resulting graphs. In this sense, the second experiment was performed. Here, the input traffic was filtered to contain only the updates that correspond to the selected RIR. This helped to provide clearer results and a better understanding of the dynamic based on the region of the collector and the inspected RIR.

The first two experiments were devoted to traffic that contained only IPv4 prefixes. In the last experiment, IPv6 was inspected. The results showed some similarity in the structure of the FIB tables containing IPv6 prefixes and the FIB tables containing IPv4 prefixes. There were noticable levels where most of the changes were focused (level 24 for IPv4 and level 48 for IPv6). The dynamicity of changes was more noticeable when the simulation was used as the FIB table implementation of the Tree Bitmap algorithm. This was also the case in the second experiment.

	SAVE UPDATES		DELETE UPDATES		The total number of~nodes	
Level	Updates	%	Updates	%	Min	Max
0.	0	0%	0	0%	1	1
1.	0	0%	0	0%	1	1
2.	0	0%	0	0%	2	2
3.	0	0%	0	0%	2	2
4.	0	0%	0	0%	2	2
5.	0	0%	0	0%	2	2
6.	0	0%	0	0%	3	3
7.	0	0%	0	0%	5	5
8.	0	0%	0	0%	7	7
9.	0	0%	0	0%	9	9
10.	0	0%	0	0%	9	9
11.	0	0%	0	0%	10	10
12.	0	0%	0	0%	11	11
13.	0	0%	0	0%	14	14
14.	0	0%	0	0%	17	17
15.	0	0%	0	0%	24	24
16.	0	0%	0	0%	38	38
17.	0	0%	0	0%	64	64
18.	0	0%	0	0%	112	112
19.	0	0%	0	0%	196	196
20.	0	0%	0	0%	350	350
21.	0	0%	0	0%	630	630
22.	0	0%	0	0%	1178	1178
23.	0	0%	0	0%	2265	2265
24.	0	0%	0	0%	4313	4313
25.	2	0%	1	0%	7573	7575
26.	6	1%	5	1%	11677	11681
27.	9	1%	7	1%	15693	15698
28.	9	1%	7	1%	17840	17845
29.	11	2%	9	2%	20278	20285
30.	36	5%	36	6%	20099	20106
31.	38	6%	39	7%	24310	24319
32.	41	6%	42	7%	27370	27381
33.	45	7%	44	8%	13631	13647
34.	47	7%	46	8%	15294	15311
35.	56	8%	53	9%	15992	16013
36.	69	10%	61	11%	18984	19010
37.	59	9%	58	10%	17530	17557
38.	70	10%	68	12%	20193	20228
39.	85	12%	81	14%	23501	23551
40.	119	17%	113	20%	29312	29395
41.	122	18%	119	21%	23297	23384
42.	140	20%	137	24%	26956	27060
43.	179	26%	168	30%	31232	31365
44.	242	35%	210	37%	38937	39115
45.	306	44%	248	44%	32924	33161
46.	412	60%	341	60%	40196	40522
47.	492	71%	405	72%	49998	50394
48.	594	86%	489	86%	65004	65480

Number of~updates:690|Number of~updates:566 |
Created nodes:3189 |Removed nodes:2787
min depth: 29 |min depth: 32
max depth: 48 |max depth: 48

Figure 6.17: The output of Dynamic Statistics of the FIB table from simulation whereas FIB model the Binary Trie was used, containing only IPv6 prefixes. Snapshot originating from collector RRC01.

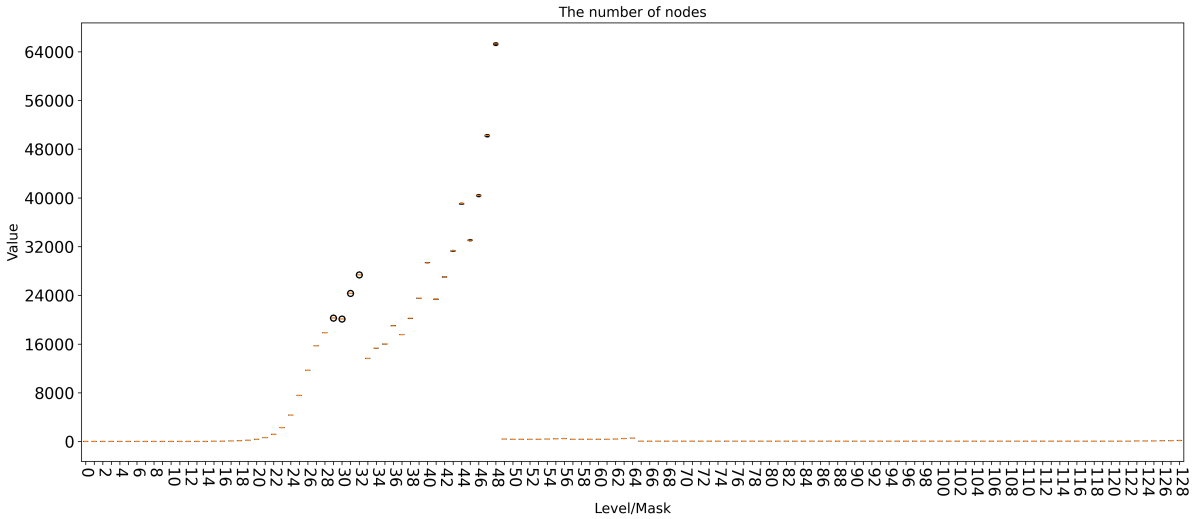

Figure 6.18: The graph of the number of nodes, generated from simulation whereas FIB model the Binary Trie algorithm was used, for only IPv6 prefixes. Snapshot originating from collector RRC01.

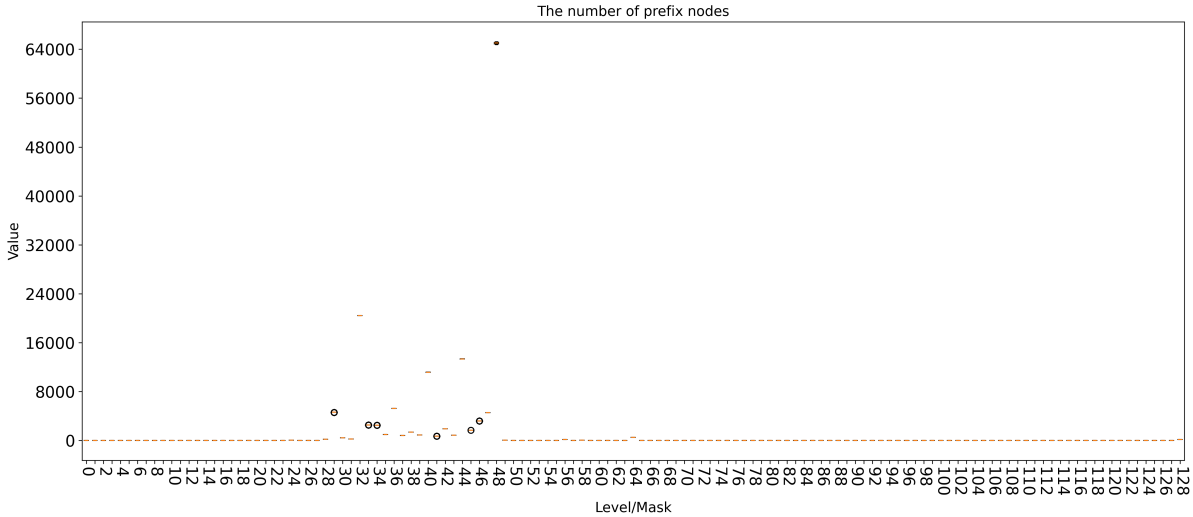

Figure 6.19: The graph of number of prefix nodes, generated from simulation whereas FIB model the Binary Trie algorithm was used, for only IPv6 prefixes. Snapshot originating from collector RRC01.

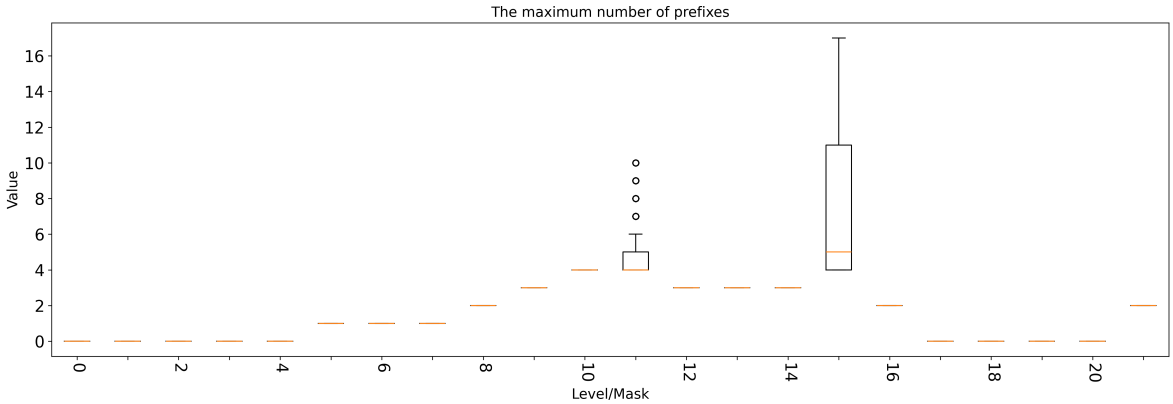

Figure 6.20: The graph of the maximum number of prefixes, generated from simulation whereas FIB model the Tree Bitmap algorithm was used, for only IPv6 prefixes. Snapshot originating from collector SOXRS.

Chapter 7

Conclusion

The objective of the thesis was to explore and develop a framework that will be able to analyze the behavior of a provided LPM algorithm, in the context of changes of data structures inside core routers. As a result of the thesis, the SandRouter framework was developed, and its functionality was later tested by experiments.

The development of SandRouter included a study of the role and management of RIB and FIB tables within routers. Then, the problem of packet classification and the mechanisms of LPM algorithms were discussed. The properties of these algorithms were used to design statistics which are going to be collected and provided as the output of SandRouter. The statistics were split into two types: Dynamic Statistics, which visualize the dynamicity of changes in the FIB table data structure, and Structural Statistics, provided in the form of graphs that vizualize the shape of data structures, the number of nodes, prefixes, etc.

The SandRouter framework provides the ability to analyze the behavior of LPM algorithms by simulating BGP traffic, which can be obtained from raw BGP data storages. The framework as a whole is a simplified model of a router, which allows to change the implementation of the FIB table. In addition, during simulation, the FIB table is monitored, and all changes are recorded. At the end of the simulation, the collected data are processed and provided in the form of Structural Statistics and Dynamic Statistics. The framework is able to process prefixes of two IP protocol types, IPv4 and IPv6. In addition, two FIB models are provided. They implement the Binary Trie algorithm and the Tree Bitmap algorithm.

To prove that SandRouter is capable of showing the dynamicity in structures of FIB tables, three experiments were carried out. The first experiment provided results from simulations, in which both FIB table models were tested. The experiment focused on IPv4 prefixes only and the tested collectors were chosen randomly. The results did not show much of the dynamicity and from this, a conclusion was made. The inspected FIB table data structure was too large and the changes made by BGP updates were too small to be visible in the final graphs. In this way, a second experiment was conducted. There, the traffic from snapshot and update data files was filtered according to the Regional Internet Registry. The idea was then further enhanced. There is a possibility to get the traffic data from different collectors, located in different regions and continents. In this sense, scripts have been developed that can preprocess the input data for simulation, based on the selected RIR. The results show that the difference in sizes of the FIB tables and dynamicity can be found based on the region of the collector and the isnpected RIR. The most visible

differences were found for the AFRINIC registry and for the LACNIC registry. The final experiment focused only on IPv6 prefixes.

In Chapter 5 the implementation of SandRouter is described. There is a description of how a new FIB model should be implemented, also with information on methods, useful objects, and predefined formats for statistics. In the README file, located in *SandRouter/* folder (see Appendix A), is described a tutorial to perform on how Experiment 2. In this way, the results described in Section 6.2 will be generated.

7.1 Future Extensions

Between possible improvements could be the following proposals.

- **Optimization of the SandRouter's implementation** At this point, some simulations can run for more than an hour, even with a small data set. Because the SandRouter framework is more proof-of-concept, there are several opportunities for optimalization.
- **Create a configuration file** To execute a simulation, at least four input arguments must be provided. This makes the commands unclear and complex. The configuration file can make the execution of SandRouter clearer.
- **Preprocessing input data implicitly by SandRouter** The execution of the SandRouter framework is carried out in several steps. These steps include the execution of dedicated scripts or the download of files. SandRouter can do this operation implicitly, so there is no need to manually prepare input data.
- **Additional input parameters** There can be implemented other user specific functionalities, such as execution of automated tests, automated downloading of raw BGP data sets from a specific source (specifying date, time and interval of updates).
- **Better quality and a format of graphs** The format and scaling of graphs may be problematic and confusing in some cases. This extension can include the optimalization of graph generation.
- **Parameterization of properties of already implemented FIB models** There can be a possibility for the user to specify, for example, SL (stride length) for the Tree Bitmap algorithm.

Bibliography

[1] *Historie národní sítě pro vědu, výzkum a vzdělání.* Available at: https://www.cesnet.cz/sdruzeni/dokumenty/historie-narodni-site-pro-vedu-vyzkum-a-vzdelavani/.

[2] *Internet Protocol* [RFC 791]. RFC Editor, september 1981. DOI: 10.17487/RFC0791. Available at: https://www.rfc-editor.org/info/rfc791.

[3] AFRINIC organisation. [web]. Accessed: April 28, 2024. Available at: https://afrinic.net/about.

[4] APNIC organisation. [web]. Accessed: April 28, 2024. Available at: https://www.apnic.net/about-apnic/organization/apnic-region/.

[5] ARIN organisation. [web]. Accessed: April 28, 2024. Available at: https://www.arin.net/about/welcome/region/.

[6] Cerf, D. V. G. *IAB recommended policy on distributing internet identifier assignment and IAB recommended policy change to internet „connected“ status* [RFC 1174]. RFC Editor, august 1990. DOI: 10.17487/RFC1174. Available at: https://www.rfc-editor.org/info/rfc1174.

[7] CESNET Association. *CESNET 3 network* [web]. Accessed: May 4, 2024. Available at: https://www.cesnet.cz/en/sit-cesnet3-eng.

[8] CISCO Systems, I. 2022. Available at: https://www.cisco.com/c/en/us/support/docs/routers/12000-series-routers/47321-ciscoef.html.

[9] Eatherton, W., Varghese, G. and Dittia, Z. Tree bitmap: hardware/software IP lookups with incremental updates. *SIGCOMM Comput. Commun. Rev.* New York, NY, USA: Association for Computing Machinery. apr 2004, vol. 34, no. 2, p. 97–122. DOI: 10.1145/997150.997160. ISSN 0146-4833. Available at: https://doi.org/10.1145/997150.997160.

[10] Eatherton, W., Varghese, G. and Dittia, Z. Tree Bitmap: Hardware/Software IP Lookups with Incremental Updates. *SIGCOMM Comput. Commun. Rev.* New York, NY, USA: Association for Computing Machinery. apr 2004, vol. 34, no. 2, p. 97–122. DOI: 10.1145/997150.997160. ISSN 0146-4833. Available at: https://doi.org/10.1145/997150.997160.

[11] El Zarki, M. *IP - The Internet Protocol.* Available at: https://ics.uci.edu/~magda/cs620/ch4.pdf.

[12] Fuller, V., Li, T., Varadhan, K. and Yu, J. *Classless Inter-Domain Routing (CIDR): an Address Assignment and Aggregation Strategy* [RFC 1519]. RFC Editor, september 1993. DOI: 10.17487/RFC1519. Available at: https://www.rfc-editor.org/info/rfc1519.

[13] Gerich, E. P. *Guidelines for Management of IP Address Space* [RFC 1466]. RFC Editor, may 1993. DOI: 10.17487/RFC1466. Available at: https://www.rfc-editor.org/info/rfc1466.

[14] Goralski, W. *The Illustrated Network, Second Edition: How TCP/IP Works in a Modern Network*. 2ndth ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2017. ISBN 0128110279.

[15] Hinden, B. *IP Version 6 Addressing Architecture* [RFC 2373]. RFC Editor, july 1998. DOI: 10.17487/RFC2373. Available at: https://www.rfc-editor.org/info/rfc2373.

[16] Hinden, B. and Deering, D. S. E. *Internet Protocol, Version 6 (IPv6) Specification* [RFC 2460]. RFC Editor, december 1998. DOI: 10.17487/RFC2460. Available at: https://www.rfc-editor.org/info/rfc2460.

[17] Hurricane Electric Information Services. *Internet Statistics* [web]. Accessed: April 28, 2024. Available at: https://bgp.he.net/report/netstats.

[18] IANA. *IPv4 Address Space* [web]. Accessed: April 29, 2024. Available at: https://www.iana.org/assignments/ipv4-address-space/ipv4-address-space.xhtml.

[19] IANA. *IPv6 Unicast Address Assignments* [web]. Accessed: April 29, 2024. Available at: https://www.iana.org/assignments/ipv6-unicast-address-assignments/ipv6-unicast-address-assignments.xhtml.

[20] ITU-T. *Information Technology - Open Systems Interconnection - Basic Reference Model: The Basic Model*. ITU-T Recommendation X.200. ITU-T, 1994. Available at: https://www.itu.int/rec/T-REC-X.200-199407-I.

[21] Kohler, Eddie and Morris, Robert and Chen, Benjie and Jannotti, John and Kaashoek, M. Franks. *The Click Modular Router*. 1999. Available at: https://pdos.csail.mit.edu/papers/click:tocs00/paper.pdf.

[22] Kurose, J. F. and Ross, K. W. *Computer Networking: A Top-Down Approach (7th Edition)*. 7thth ed. Pearson, 2012. ISBN 0132856204.

[23] LACNIC organization. *Coverage Area* [web]. Accessed: April 28, 2024. Available at: https://www.lacnic.net/631/2/lacnic/coverage-area.

[24] Matoušek, J. *Addressing Issues in Research on Packet Classification in Core Networks*. Brno, CZ, 2019. Disertační práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Available at: https://www.fit.vut.cz/study/phd-thesis/642/.

[25] Medhi, D. and Ramasamy, K. *Network Routing: Algorithms, Protocols, and Architectures*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2007. ISBN 0120885883.

[26] Mir, N. F. *Computer and Communication Networks*. USA: Prentice Hall PTR, 2006. ISBN 0131747991.

[27] Rekhter, Y., Hares, S. and Li, T. *A Border Gateway Protocol 4 (BGP-4)* [RFC 4271]. RFC Editor, january 2006. DOI: 10.17487/RFC4271. Available at: https://www.rfc-editor.org/info/rfc4271.

[28] RIPE NCC. [web]. Accessed: April 4, 2024. Available at: https://www.ripe.net/.

[29] RouteViews group. *RouteViews* [web]. Accessed: May 7, 2024. Available at: https://www.routeviews.org/routeviews/index.php/about/.

[30] Song, H., Turner, J. and Lockwood, J. Shape shifting tries for faster IP route lookup. In: *13TH IEEE International Conference on Network Protocols (ICNP'05)*. 2005, p. 10 pp.–367. DOI: 10.1109/ICNP.2005.36.

Appendix A

Content of the Storage Medium